



INSTITUTO SUPERIOR POLITÉCNICO GAYA

Licenciatura em Engenharia Informática

Projeto de Engenharia Informática em Contexto Empresarial

BoxToCar

SISTEMA DE PICKING HUMANO-MÁQUINA: INDÚSTRIA 5.0

Luís Miguel Marques Rafael
2023103849

*Relatório de estágio/projeto orientado pelo Professor Fernando Luís Ferreira de Almeida
e apresentado à Escola Superior de Ciência e Tecnologia*

Vila Nova de Gaia
Junho de 2026

Dedicatória

*À minha família, pelo apoio incondicional
e por acreditarem em mim em cada etapa deste percurso.
Aos meus amigos, pela presença nos momentos certos.*

Agradecimentos

Agradeço, em primeiro lugar, à minha família, por todos os valores e ensinamentos transmitidos e pelo apoio constante ao longo de todo o percurso académico.

Ao ISPGAYA e, em particular, ao Professor Fernando Luís Ferreira de Almeida, orientador deste projeto, pela disponibilidade, orientação e rigor com que acompanhou o trabalho.

A todos os docentes da Licenciatura em Engenharia Informática, pelo contributo determinante na minha formação, e aos colegas, hoje amigos, que tornaram esta jornada mais leve e memorável.

Resumo

O presente relatório descreve o desenvolvimento do BoxToCar, uma prova de conceito de um sistema de picking colaborativo humano-máquina inscrito no paradigma da Indústria 5.0, realizado no âmbito do projeto final da Licenciatura em Engenharia Informática. Ao contrário das abordagens que procuram substituir o operador, o sistema explora a complementaridade entre o ser humano - responsável pela destreza da recolha - e uma frota de robôs móveis (AMR) com especificações físicas distintas - capacidade de carga, velocidade e tipo de navegação -, coordenados por um núcleo lógico que os faz convergir em pontos de encontro eficientes.

A solução é composta por três componentes integrados: uma API REST (Node.js/Express) que concentra a lógica de negócio e a coordenação; um backoffice web para o gestor, com um mapa interativo do armazém, um catálogo de robôs com recomendação automática de acordo com o layout do armazém, gestão de stock e visualização da coordenação; e uma aplicação móvel (Flutter) que guia o operador até à prateleira e apresenta a atribuição decidida para cada recolha.

A atribuição de cada recolha ao operador ou ao robô resulta de uma decisão multicritério - que combina o tempo de deslocação de cada agente, a velocidades distintas, com penalizações associadas ao peso e à fragilidade do artigo face à capacidade do robô selecionado - e não apenas da distância percorrida. A atribuição e a sequência das recolhas pendentes de um armazém são otimizadas por três algoritmos complementares: um algoritmo exato, por força bruta, viável apenas para conjuntos pequenos; uma heurística construtiva gulosa; e uma meta-heurística de recozimento simulado que parte da solução gulosa e a refina por troca de atribuições e de ordens. Os três algoritmos foram comparados quanto ao tempo de cálculo e à qualidade da solução obtida. O cálculo de distâncias e rotas sobre a grelha do armazém recorre a uma procura em largura (BFS), que serve de utilitário de apoio aos três algoritmos e já não constitui, por si só, o mecanismo de decisão.

O sistema foi validado através de testes funcionais às operações críticas, de testes unitários ao módulo de coordenação e de testes comparativos aos três algoritmos de planeamento, em vários cenários de armazém. A arquitetura assenta em autenticação por tokens JWT, palavras-passe protegidas por bcrypt, um código de convite para o registo de operadores e isolamento de dados entre armazéns, com um modelo de dados em que produtos e encomendas se ligam aos artigos por chave estrangeira. O sistema valida ainda o encaixe físico de cada artigo na prateleira e recusa construir ou importar layouts com prateleiras inacessíveis, mantendo a largura do corredor, o tamanho das prateleiras e o robô de cada armazém configuráveis pelo gestor.

Palavras-chave: Indústria 5.0; Picking Colaborativo; Robôs Móveis Autónomos; Decisão Multicritério; Heurísticas; Meta-heurísticas; Sistema de Apoio à Decisão; API REST; Flutter; Coordenação Humano-Máquina.

Abstract

This report describes the development of BoxToCar, a proof of concept of a collaborative human-machine picking system framed within the Industry 5.0 paradigm, carried out as the final project of the Bachelor's Degree in Computer Engineering. Rather than replacing the operator, the system exploits the complementarity between the human - responsible for the dexterity of picking - and a fleet of mobile robots (AMR) with distinct physical specifications - load capacity, speed and navigation type -, coordinated by a logical core that makes them converge at efficient meeting points.

The solution comprises three integrated components: a REST API (Node.js/Express) concentrating the business logic and coordination; a web backoffice for the manager, with an interactive warehouse map, a robot catalogue with automatic recommendation based on the warehouse layout, stock management and coordination visualisation; and a mobile application (Flutter) that guides the operator to the shelf and displays the assignment decided for each pick.

The assignment of each pick to the operator or to the robot results from a multicriteria decision - combining the travel time of each agent, at distinct speeds, with penalties for weight and fragility relative to the selected robot's capacity - rather than distance alone. The assignment and sequencing of a warehouse's pending picks are optimised by three complementary algorithms: an exact brute-force algorithm, viable only for small sets; a greedy constructive heuristic; and a simulated-annealing metaheuristic that starts from the greedy solution and refines it by swapping assignments and orderings. The three algorithms were compared in terms of computation time and solution quality. The computation of distances and routes over the warehouse grid relies on a breadth-first search (BFS), which serves as a supporting utility for the three algorithms rather than the decision mechanism itself.

The system was validated through functional tests of the critical operations, unit tests of the coordination module, and comparative tests of the three planning algorithms, across several warehouse scenarios. The architecture relies on JWT token authentication, passwords protected with bcrypt, an invite code for operator registration, and data isolation between warehouses, with a data model in which products and orders reference articles by foreign key. The system also validates the physical fit of each article on the shelf and refuses to build or import layouts with unreachable shelves, keeping the corridor width, shelf size and the robot of each warehouse configurable by the manager.

Keywords: Industry 5.0; Collaborative Picking; Autonomous Mobile Robots; Multicriteria Decision; Heuristics; Metaheuristics; Decision Support System; REST API; Flutter; Human-Machine Coordination.

Lista de Abreviaturas e Siglas

AGV - Automated Guided Vehicle (Veículo de Guiado Automático)
AMR - Autonomous Mobile Robot (Robô Móvel Autônomo)
API - Application Programming Interface
APA - American Psychological Association
BFS - Breadth-First Search (Procura em Largura)
ERP - Enterprise Resource Planning
HTTP - HyperText Transfer Protocol
ISPGAYA - Instituto Superior Politécnico Gaya
JSON - JavaScript Object Notation
JWT - JSON Web Token
PME - Pequena e Média Empresa
REST - Representational State Transfer
SGBD - Sistema de Gestão de Bases de Dados
SKU - Stock Keeping Unit
SQL - Structured Query Language
WMS - Warehouse Management System (Sistema de Gestão de Armazém)

Índice

Dedicatória	2
Agradecimentos	3
Resumo	4
Abstract.....	5
Lista de Abreviaturas e Siglas	6
Índice	7
Índice de Apêndices	10
Índice de Figuras	11
Índice de Tabelas.....	12
1. Introdução	13
1.1. Contexto e motivação.....	13
1.2. Objetivos.....	13
1.3. Estrutura do documento.....	14
1.4. Declaração de utilização de ferramentas de inteligência artificial.....	14
2. Caracterização da Instituição.....	15
3. Enquadramento e Análise de Necessidades.....	16
3.1. Contexto Institucional.....	16
3.2. Análise de Necessidades.....	16
4. Análise de Soluções Disponíveis no Mercado	17
4.1. Sistemas de gestão de armazém tradicionais	17
4.2. Sistemas colaborativos e robôs móveis autónomos.....	17
4.3. Comparação de soluções.....	17
4.4. Limitações e justificação do desenvolvimento próprio.....	17
5. Bases Teóricas.....	19
5.1. Indústria 5.0 e a colaboração humano-máquina	19
5.2. Robôs móveis autónomos (AMR) na intralogística.....	19
5.3. Seleção do tipo de robô e sistemas de apoio à decisão	19
5.4. Sistemas WMS e o processo de picking.....	20
5.5. Estratégias de picking colaborativo.....	20
5.6. Arquitetura cliente-servidor e APIs REST.....	21
5.7. Algoritmos de procura de caminho em grelha	21
5.8. O problema do ponto de encontro (rendezvous)	22
5.9. Alocação de tarefas humano-robô: complexidade e heurísticas	23
5.10. Autenticação (JWT) e hashing de palavras-passe (bcrypt).....	23

5.11.	Caracterização física dos artigos e o seu papel na decisão	23
5.12.	Persistência relacional e desenvolvimento multiplataforma.....	24
6.	Análise, Arquitetura e Planeamento.....	25
6.1.	Requisitos do sistema	25
6.1.1.	Requisitos não funcionais	26
6.2.	Arquitetura da solução	27
6.3.	Modelo de dados.....	28
6.4.	Validação física de encaixe e de acessibilidade	29
6.5.	Planeamento e metodologia.....	30
7.	Manual de Utilizador.....	31
7.1.	Introdução	31
7.2.	Guia do Gestor (Backoffice Web)	31
7.2.1.	Criar conta e armazém	31
7.2.2.	Iniciar sessão.....	31
7.2.3.	O painel e o mapa do armazém.....	31
7.2.4.	Definir o tamanho e construir prateleiras.....	31
7.2.5.	Configurar o armazém	31
7.2.6.	Dar entrada de produtos.....	32
7.2.7.	Gerar ordens de picking	32
7.2.8.	Planear a coordenação humano-máquina.....	32
7.3.	Guia do Operador (Aplicação Móvel).....	32
7.3.1.	Criar conta e iniciar sessão.....	32
7.3.2.	Executar uma recolha.....	32
7.4.	Resolução de Problemas (FAQ)	33
8.	Autenticação, Autorização e Multi-armazém	34
9.	Gestão do Armazém e Mapa Interativo	35
10.	Gestão de Stock e Ordens de Picking	36
11.	Coordenação Humano-Máquina	37
11.1.	Modelo de coordenação	37
11.2.	Recomendação de robô (Sistema de Apoio à Decisão).....	37
11.3.	Cálculo do ponto de encontro	37
11.4.	Decisão multicritério humano-robô	38
11.5.	Os três algoritmos de planeamento	38
11.6.	Implementação.....	40
11.7.	Visualização.....	40
11.8.	Métricas de eficiência.....	41
12.	Aplicação Móvel do Operador.....	42

13.	Testes de Validação	43
13.1.	Metodologia de teste.....	43
13.2.	Testes funcionais.....	43
13.3.	Testes ao algoritmo de coordenação.....	44
13.4.	Testes unitários ao módulo de coordenação	45
13.5.	Testes comparativos aos três algoritmos de planeamento	45
13.6.	Análise dos resultados.....	46
14.	Sítio Web do Projeto.....	47
15.	Conclusões.....	47
15.1.	Resultados obtidos.....	47
15.2.	Dificuldades e soluções	47
15.3.	Melhorias futuras	47
15.4.	Limitações.....	48
15.5.	Contributo pessoal e aprendizagens	48
	Referências Bibliográficas.....	49
	Glossário	51
	Apêndices.....	52
	Apêndice A - Estrutura de rotas da API.....	52
	Apêndice B - Esquema das tabelas da base de dados.....	52
	Apêndice C - Pseudocódigo dos algoritmos de coordenação e planeamento.....	52
	Apêndice D - Declaração de utilização de ferramentas de inteligência artificial	53
	Prompt:.....	54

Índice de Apêndices

Apêndice A - Estrutura de rotas da API

Apêndice B - Esquema das tabelas da base de dados

Apêndice C - Pseudocódigo do algoritmo de coordenação

Apêndice D - Declaração de utilização de ferramentas de inteligência artificial

Índice de Figuras

Figura 1 - Diagrama de casos de uso.....	24
Figura 2 - Arquitetura cliente-servidor da solução.	25
Figura 3 - Modelo de dados (diagrama entidade-relação).....	25
Figura 4 - Ecrã de início de sessão do backoffice web.	31
Figura 5 - Backoffice web com o mapa interativo do armazém.....	32
Figura 6 - Entrada de produtos e simulador de ordens de picking (ERP).....	33
Figura 7 - Fluxo do processo de picking.....	33
Figura 8 - Coordenação humano-máquina: ponto de encontro e rotas calculadas.....	37
Figura 9 - Aplicação móvel do operador: tarefa e ponto de encontro com o robô	38
Figura 10 - Diagrama de sequência	39

Índice de Tabelas

Tabela 1 - Comparação entre soluções de mercado e o BoxToCar.....	17
Tabela 2 - Requisitos funcionais e respetiva prioridade.....	25
Tabela 3 - Tabelas principais da base de dados.....	29
Tabela 4 - Comparação dos três algoritmos de planeamento (armazém 7×6, 6 recolhas).	40
Tabela 5 - Casos de teste funcionais e resultados obtidos.	44
Tabela 6 - Resultados do algoritmo de coordenação (armazém 7×6).	44
Tabela 7 - Resultados do algoritmo de coordenação num segundo layout (5×7).	45
Tabela 8 - Comparação dos três algoritmos (armazém 7×6, 6 recolhas, robô AMR Ligeiro).	45
Tabela 9 - Comparação dos três algoritmos num segundo layout (5×7, 5 recolhas, robô AMR Pesado).	46

1. Introdução

1.1. Contexto e motivação

A quarta revolução industrial (Indústria 4.0) centrou-se na automação, na conectividade e na digitalização dos processos produtivos. Mais recentemente, o conceito de Indústria 5.0 tem vindo a complementar essa visão, recolocando o ser humano no centro do sistema produtivo e valorizando a colaboração - e não a substituição - entre pessoas e máquinas, com objetivos de resiliência, sustentabilidade e bem-estar do trabalhador (Xu et al., 2021; Leng et al., 2022).

A intralogística, e em particular a operação de armazém, é um domínio onde este paradigma tem aplicação direta. O picking - a recolha física dos artigos que compõem uma encomenda - é reconhecido como uma das operações mais críticas e dispendiosas de um armazém, sobretudo pelo tempo despendido em deslocações e pelo esforço físico associado ao transporte de carga (Van Gils et al., 2018; Boysen et al., 2019). A introdução de robôs móveis autónomos (AMR) na intralogística tem-se revelado uma via eficaz para aumentar a produtividade e reduzir o esforço humano (Fragapane et al., 2021).

É neste contexto que se enquadra o BoxToCar. Em vez de automatizar totalmente a recolha - o que exigiria manipuladores complexos e dispendiosos -, o sistema adota uma abordagem colaborativa: o operador humano executa a recolha de precisão (destreza), enquanto um robô virtual assume o transporte da carga ao longo de rotas fixas, encontrando-se com o operador em pontos estratégicos. A motivação do projeto reside em demonstrar, através de uma prova de conceito funcional, a viabilidade e o valor deste modelo de coordenação humano-máquina.

1.2. Objetivos

Em conformidade com a proposta de projeto, o objetivo central é o desenvolvimento de uma prova de conceito de um sistema logístico colaborativo (Indústria 5.0) que coordene, de forma lógica, um operador humano e um robô virtual na preparação de encomendas. Foram definidos os seguintes objetivos específicos:

- Desenvolver um protótipo de software composto por Backend (API), Backoffice Web e Aplicação Móvel.
- Implementar a importação dinâmica de layouts de armazém em grelha 2D através de ficheiros estruturados (JSON).
- Criar um sistema lógico de coordenação que sincronize as tarefas de recolha do operador com o transporte de um robô virtual (AMR).

- Aplicar algoritmos exatos de cálculo de rotas para definir pontos de encontro eficientes entre o humano e a máquina, reduzindo a distância percorrida pelo operador com carga.
- Garantir a segurança do sistema (autenticação, cifragem) e o isolamento de dados entre múltiplos armazéns.

1.3. Estrutura do documento

Para além desta introdução, o documento organiza-se em mais catorze capítulos. Os capítulos 2 e 3 apresentam o enquadramento institucional e a análise de necessidades. O capítulo 4 analisa as soluções de mercado. O capítulo 5 desenvolve, com profundidade, os fundamentos teóricos (Indústria 5.0, AMR, WMS, algoritmos de rota e ponto de encontro). O capítulo 6 descreve a análise, a arquitetura e o planeamento. O capítulo 7 é um manual de utilizador do sistema. Os capítulos 8 a 12 detalham a implementação das várias componentes, com destaque para o capítulo 11, dedicado à coordenação humano-máquina. O capítulo 13 apresenta os testes de validação, o capítulo 14 descreve o sítio web do projeto e o capítulo 15 as conclusões. Seguem-se as referências, o glossário e os apêndices.

1.4. Declaração de utilização de ferramentas de inteligência artificial

No desenvolvimento deste projeto foram utilizadas ferramentas de assistência baseadas em inteligência artificial, cuja utilização se declara para efeitos de transparência.

O apoio incidiu sobre a utilização do sistema de controlo de versões Git e da plataforma GitHub, com os quais o autor tinha pouca experiência prévia, bem como sobre a revisão crítica, a correção linguística e a formatação do texto deste relatório, incluindo a verificação das referências bibliográficas segundo as normas APA e a construção da página web do projeto.

A conceção do sistema, a definição do algoritmo de coordenação humano-máquina, o desenho da arquitetura e do modelo de dados, a implementação e a validação experimental dos resultados são da inteira responsabilidade do autor. Todo o conteúdo produzido com o apoio destas ferramentas foi revisto e verificado antes de ser incorporado no relatório.

O detalhe das ferramentas utilizadas, das respetivas versões, dos objetivos, dos prompts submetidos e dos resultados obtidos encontra-se descrito no Apêndice D.

2. Caracterização da Instituição

O projeto BoxToCar foi desenvolvido no âmbito da unidade curricular de Projeto de Engenharia Informática em Contexto Empresarial, do 3.º ano da Licenciatura em Engenharia Informática do Instituto Superior Politécnico Gaya (ISPGAYA), entidade promotora e acolhedora do trabalho. O ISPGAYA é uma instituição de ensino superior sediada em Vila Nova de Gaia, vocacionada para a formação técnica e científica nas áreas da engenharia e da tecnologia, com forte ligação ao tecido empresarial e à aplicação prática do conhecimento.

O cenário de aplicação considerado é o de uma organização com um ou mais armazéns, na qual um gestor administra o espaço e o stock e um conjunto de operadores executa as recolhas, apoiados por robôs móveis de transporte. Esta caracterização orientou tanto a modelação dos dados como o desenho das interfaces e da lógica de coordenação.

O domínio escolhido - o picking colaborativo - revelou-se particularmente rico do ponto de vista técnico, por reunir num único sistema o desenho de uma API, a modelação de uma base de dados, a construção de interfaces web e móveis, a implementação de mecanismos de segurança e, sobretudo, a aplicação de algoritmos de otimização de rotas e coordenação.

3. Enquadramento e Análise de Necessidades

3.1. Contexto Institucional

A unidade curricular de Projeto tem como propósito a aplicação integrada dos conhecimentos da licenciatura na conceção de uma solução de software completa, da análise de requisitos à validação. O BoxToCar inscreve-se numa linha de trabalhos orientados para a resolução de problemas reais de intralogística, procurando materializar, num protótipo, os princípios de colaboração humano-máquina que caracterizam a Indústria 5.0.

3.2. Análise de Necessidades

A recolha de encomendas em armazém combina duas exigências distintas: a destreza para localizar e manusear artigos, na qual o ser humano é insuperável, e o transporte de carga por longas distâncias, tarefa fisicamente exigente e propensa a fadiga e lesões músculo-esqueléticas (Gualtieri et al., 2021). As soluções tradicionais tendem a colocar todo o esforço no operador, que percorre repetidamente longos trajetos a empurrar carros de carga.

Da análise destas necessidades resulta a oportunidade para um sistema que separe estas duas responsabilidades: manter o operador na tarefa de precisão e delegar o transporte pesado a um robô, coordenando ambos de forma a que a distância percorrida com carga pelo humano seja mínima. É a esta necessidade - reduzir a distância que o operador percorre com carga sem comprometer a flexibilidade da recolha manual - que o BoxToCar procura responder.

4. Análise de Soluções Disponíveis no Mercado

4.1. Sistemas de gestão de armazém tradicionais

As soluções de gestão de armazém mais difundidas surgem como módulos de sistemas ERP (por exemplo, SAP EWM ou Odoo) ou como plataformas WMS especializadas. Oferecem funcionalidades abrangentes de inventário, localização, expedição e relatórios, sendo adequadas a operações de média e grande dimensão, mas caracterizam-se por custos de licenciamento elevados e complexidade de configuração significativa (Boysen et al., 2019).

4.2. Sistemas colaborativos e robôs móveis autónomos

Uma segunda geração de soluções integra robôs móveis autónomos no fluxo de picking. Modelos como o goods-to-person (o robô traz a prateleira ao operador, popularizado pela Kiva/Amazon Robotics) ou o picker-following (o robô acompanha o operador) demonstram ganhos de produtividade assinaláveis (Azadeh et al., 2019; Fragapane et al., 2021). Estas soluções são, contudo, dispendiosas e assentam em frotas físicas com navegação autónoma e prevenção de colisões em tempo real.

Entre as variantes documentadas contam-se ainda os pontos de consolidação, em que o operador deposita os artigos recolhidos em locais de transferência a partir dos quais o robô assume o transporte - modelo de que a abordagem deste projeto se aproxima (Azadeh et al., 2019).

4.3. Comparação de soluções

A Tabela 1 compara, de forma sintética, estas categorias com a abordagem do BoxToCar, considerando critérios relevantes para um contexto de PME ou de prova de conceito académica.

Critério	WMS/ERP tradicional	Sistemas com AMR (mercado)	BoxToCar
Custo	Elevado	Muito elevado	Nulo (protótipo próprio)
Complexidade	Alta	Muito alta	Baixa
Colaboração humano-máquina	Inexistente	Sim (frota física)	Sim (robô virtual)
Otimização de ponto de encontro	Não	Parcial/proprietária	Sim (algoritmo exato)
Adequação a PME/académico	Baixa	Baixa	Alta

Tabela 1 - Comparação entre soluções de mercado e o BoxToCar.

4.4. Limitações e justificação do desenvolvimento próprio

As soluções de mercado, embora robustas, são inacessíveis a organizações de pequena dimensão e, por serem produtos fechados, não permitem explorar nem demonstrar os mecanismos internos de coordenação. Conforme referido na proposta, a componente robótica do BoxToCar é puramente virtual e opera em rotas fixas e previsíveis, o que permite demonstrar a lógica de um sistema colaborativo moderno sem incorrer na complexidade da navegação autónoma e da prevenção de colisões em tempo real. Justifica-se, assim, o desenvolvimento próprio como via para compreender, controlar e validar toda a cadeia - da base de dados ao algoritmo de coordenação.

5. Bases Teóricas

Este capítulo aprofunda os fundamentos que sustentam as opções técnicas do projeto, desde o paradigma da Indústria 5.0 e os robôs móveis autónomos até aos algoritmos de procura de caminho e ao problema do ponto de encontro.

5.1. Indústria 5.0 e a colaboração humano-máquina

A Indústria 5.0 é frequentemente descrita como uma evolução, e não uma rutura, face à Indústria 4.0: mantém a base tecnológica (automação, dados, conectividade) mas acrescenta-lhe uma orientação humano-cêntrica, sustentável e resiliente (Xu et al., 2021; Leng et al., 2022). Um dos seus pilares é a colaboração entre humanos e máquinas, na qual as capacidades complementares de ambos são exploradas: a flexibilidade cognitiva e a destreza do ser humano, e a força, a repetibilidade e a resistência da máquina (Maddikunta et al., 2022). Aplicado ao picking, este princípio traduz-se em atribuir ao operador a recolha de precisão e ao robô o transporte de carga, reduzindo a fadiga e o risco de lesões associados ao esforço físico repetido (Gualtieri et al., 2021).

5.2. Robôs móveis autónomos (AMR) na intralogística

Os robôs móveis autónomos (AMR) distinguem-se dos tradicionais veículos de guiado automático (AGV) pela sua capacidade de decisão e navegação flexível; contudo, em ambientes estruturados, é frequente - e vantajoso - restringir a sua circulação a corredores principais e rotas fixas, aumentando a previsibilidade e a segurança (Fragapane et al., 2021). No BoxToCar, o robô é modelado como um AMR virtual que circula exclusivamente pelo corredor principal (o perímetro da grelha), o que dispensa a modelação de navegação autónoma e de prevenção de colisões, mantendo o foco na lógica de coordenação.

A literatura mais recente sublinha que a escolha do robô não pode ser genérica: cada tipologia - desde veículos de guiado automático (AGV) de trilho fixo até AMR de

navegação livre, passando por soluções de navegação por perímetro - tem capacidade de carga, velocidade e requisitos de espaço de manobra distintos, que devem ser casados com o layout concreto do armazém e com o perfil do stock a movimentar (Lackner et al., 2024). É esta análise prévia de compatibilidade física - e não apenas a disponibilidade orçamental - que deve orientar a seleção do robô, sob pena de sobredimensionar (custo e espaço desperdiçados) ou subdimensionar (incapacidade de transportar os artigos mais pesados) a frota.

5.3. Seleção do tipo de robô e sistemas de apoio à decisão

A decisão sobre qual o tipo de robô operar num armazém concreto configura, ela própria, um problema de apoio à decisão: dado um layout - nomeadamente a largura dos corredores - e o perfil de carga do stock a movimentar, importa filtrar os robôs fisicamente compatíveis e, de entre estes, recomendar o que melhor equilibra capacidade e velocidade sem sobredimensionar o investimento (Calzavara et al., 2023). O BoxToCar incorpora este raciocínio num módulo de recomendação: mantém um catálogo de tipos de robô - cada um com capacidade de carga, velocidade, tipo de navegação (trilho fixo, perímetro ou livre) e a largura mínima de corredor que exige - e, para cada armazém, filtra os tipos compatíveis com o seu layout e pontua-os pelo ajuste entre a capacidade e a carga típica das recolhas pendentes, comportando-se como um sistema de apoio à decisão (SAD) e não como a escolha fixa e arbitrária de um único robô.

A Tabela 2 sistematiza o catálogo atual, resultado desta análise prévia de tipologias.

Robô	Navegação	Capacidade (kg)	Velocidade (m/s)	Corredor mínimo (m)	Dimensões C×L×A (cm)
AGV Trilho Fixo	Trilho fixo	40	1,50	1,00	80×60×120
AMR Ligeiro	Perímetro	15	1,20	0,80	50×40×100
AMR Pesado	Perímetro	80	0,80	1,20	100×80×150
Cobot Colaborativo	Livre	25	0,60	0,60	45×45×110

Tabela 2 - Catálogo de tipologias de robô e as suas especificações.

O catálogo regista ainda as dimensões físicas de cada robô - comprimento, largura e altura -, que dão sentido à largura mínima de corredor exigida por cada tipo e servem de base a futuras verificações de encaixe entre o robô e os corredores ou portas do armazém.

5.4. Sistemas WMS e o processo de picking

Um sistema de gestão de armazém (WMS) apoia e controla as operações de receção, armazenamento, localização e expedição, procurando maximizar a utilização do

espaço e a exatidão do inventário. Dentro destas operações, o picking é aquela em que a deslocação tem maior peso; a literatura recente sobre desenho de sistemas de picking sublinha precisamente a importância de reduzir as distâncias percorridas e de coordenar recursos (Van Gils et al., 2018; Boysen et al., 2019). O BoxToCar atua sobre esta dimensão, minimizando a distância que o operador percorre com carga.

5.5. Estratégias de picking colaborativo

Entre as estratégias de picking assistido por robôs contam-se o goods-to-person, o picker-following e os pontos de consolidação, nos quais o operador deposita os artigos recolhidos em pontos de transferência a partir dos quais o robô assume o transporte (Azadeh et al., 2019). O modelo adotado neste projeto aproxima-se deste último: define-se, para cada recolha, um ponto de encontro no corredor principal onde o operador entrega a carga ao robô, minimizando o percurso manual com peso.

Trabalhos mais recentes sobre a coordenação entre operadores humanos e frotas de AMR em picking, como o de Löffler et al. (2023), tratam explicitamente a atribuição e o encaminhamento de múltiplos robôs e múltiplos operadores como um problema de otimização combinatória, em que as decisões de atribuição e de sequenciamento se condicionam mutuamente - e não como um cálculo de distância isolado por recolha. O BoxToCar segue esta linha: a atribuição de cada recolha ao operador ou ao robô, e a ordem pela qual as recolhas pendentes são executadas, são tratadas em conjunto pelos três algoritmos de planeamento descritos no capítulo 11.

5.6. Arquitetura cliente-servidor e APIs REST

A solução adota uma arquitetura cliente-servidor, na qual um servidor central concentra a lógica de negócio e o acesso aos dados, e vários clientes (backoffice web e aplicação móvel) consomem os seus serviços. A comunicação segue o estilo arquitetural REST, proposto por Fielding (2000), que utiliza os métodos do protocolo HTTP e a troca de representações em JSON, promovendo a separação de responsabilidades, a ausência de estado no servidor e a reutilização da mesma lógica por diferentes clientes.

5.7. Algoritmos de procura de caminho em grelha

O cálculo de rotas num armazém modelado como grelha 2D é um problema clássico de procura de caminho mais curto num grafo. Três algoritmos são de referência (Cormen et al., 2009): a procura em largura (Breadth-First Search, BFS), que garante o caminho mais curto em número de passos quando as arestas têm custo unitário; o algoritmo de Dijkstra, que generaliza a BFS para arestas com custos não negativos; e o algoritmo A*, que acelera a procura recorrendo a uma heurística admissível (por exemplo, a distância de Manhattan) para orientar a expansão (Hart et al., 1968). Numa grelha uniforme, na qual todos os movimentos têm custo unitário, a BFS é exata e suficiente, pelo que foi a estratégia adotada. Caso se introduzam custos

diferenciados - por exemplo, corredores de trânsito mais rápido -, a BFS deixa de garantir o caminho de menor custo, sendo o algoritmo de Dijkstra o adequado; o algoritmo A* acrescenta-lhe uma heurística admissível, que reduz o número de células exploradas sem comprometer a otimalidade da solução. Importa notar que uma simples distância geométrica (como a distância de Manhattan até ao corredor) seria insuficiente: as prateleiras são obstáculos e uma medida em linha reta ignoraria as "paredes", conduzindo a caminhos inválidos. É precisamente a presença de obstáculos que obriga a uma procura sobre a grelha, em vez de uma fórmula fechada.

5.8. O problema do ponto de encontro (rendezvous)

A coordenação entre o operador e o robô configura uma variante do problema do ponto de encontro (rendezvous): dados dois agentes com origens distintas, determinar um ponto de convergência que otimize um dado critério. Neste projeto, o critério é assimétrico - pretende-se minimizar a distância percorrida pelo operador com carga (o recurso mais oneroso e sujeito a fadiga), aceitando que o robô, que não se cansa, percorra distâncias maiores. O ponto de encontro é, por isso, a célula do corredor principal mais próxima da prateleira, obtida por uma procura em largura a partir dos acessos à prateleira.

5.9. Alocação de tarefas humano-robô: complexidade e heurísticas

A atribuição de um conjunto de recolhas a um operador humano e a um robô, e a definição da ordem pela qual são executadas, é uma variante do problema de escalonamento e alocação de tarefas em sistemas colaborativos humano-robô, cuja complexidade combinatória cresce rapidamente com o número de tarefas - o espaço de procura inclui tanto as ordens possíveis, de crescimento fatorial, como as atribuições possíveis por tarefa -, razão pela qual a literatura recorre sistematicamente a heurísticas e meta-heurísticas, e não a algoritmos exatos, para instâncias de dimensão realista (Petzoldt et al., 2023). Entre as abordagens documentadas contam-se heurísticas construtivas gulosas, que decidem cada atribuição localmente sem garantir o ótimo, e meta-heurísticas - como algoritmos genéticos ou recozimento simulado (Kirkpatrick et al., 1983) - que partem de uma solução inicial e a refinam por perturbação sucessiva, sendo particularmente utilizadas em contextos de Indústria 5.0 para equilibrar produtividade, esforço do operador e restrições físicas dos robôs (Calzavara et al., 2023). O BoxToCar segue esta prática: compara um algoritmo exato - viável apenas para poucas tarefas -, uma heurística gulosa e uma meta-heurística de recozimento simulado (capítulo 11).

5.10. Autenticação (JWT) e hashing de palavras-passe (bcrypt)

A autenticação baseia-se em JSON Web Tokens (JWT), um formato compacto e autossuficiente definido na RFC 7519 (Jones et al., 2015): após validar as credenciais, o servidor emite um token assinado que o cliente envia em cada pedido, permitindo

autorização sem estado de sessão. As palavras-passe são protegidas com a função de hashing bcrypt (Provos & Mazières, 1999), que aplica um fator de custo configurável e um salt único por registo, tornando inviável a sua recuperação em caso de fuga.

5.11. Caracterização física dos artigos e o seu papel na decisão

Para além das especificações do robô, a decisão de atribuição depende também das características físicas do artigo a movimentar: o peso, que pode exceder a capacidade de carga do robô selecionado, tornando essa opção fisicamente inviável e não apenas desvantajosa; e a fragilidade, que introduz um risco acrescido de dano no manuseamento e na entrega no ponto de encontro, penalizando a opção de transporte por robô mesmo quando esta seria, à partida, mais rápida (Gualtieri et al., 2021). O BoxToCar regista estes atributos - peso, dimensões e fragilidade - como dados mestre de cada artigo, e não do robô ou do operador, permitindo que a decisão de atribuição os pondere de forma sistemática em cada recolha.

Para além da capacidade do robô, o BoxToCar valida também se o artigo cabe fisicamente na prateleira: todas as prateleiras de um armazém têm o mesmo tamanho - largura, profundidade e altura por nível, definidos pelo gestor -, e a entrada de stock testa as duas orientações possíveis do artigo no plano horizontal, permitindo rodá-lo, antes de a aceitar. Um artigo maior do que a prateleira, em qualquer orientação, é recusado na origem, em vez de ser aceite e só falhar mais tarde, já na recolha.

5.12. Persistência relacional e desenvolvimento multiplataforma

A persistência é assegurada por um sistema de gestão de bases de dados relacional (MySQL), que organiza a informação em tabelas relacionadas por chaves, com consultas parametrizadas que previnem injeções de SQL e um pool de ligações para desempenho. A única exceção é a cláusula LIMIT de uma operação de baixa de stock que, por não ser parametrizável, recebe um valor previamente convertido para inteiro, evitando a concatenação direta de texto não validado (limitação retomada na secção 15.4). A aplicação do operador foi desenvolvida em Flutter, framework que, a partir de uma única base de código em Dart, gera aplicações para várias plataformas, reduzindo o esforço de desenvolvimento (Google, 2023).

6. Análise, Arquitetura e Planejamento

6.1. Requisitos do sistema

Os requisitos funcionais foram derivados diretamente dos objetivos da proposta e encontram-se, com a respectiva prioridade, na Tabela 2.

Ref.	Requisito funcional (conforme a proposta)	Prioridade
RF1	Protótipo com Backend (API), Backoffice Web e Aplicação Móvel	Alta
RF2	Importação dinâmica de layouts de armazém em grelha 2D (JSON)	Alta
RF3	Autenticação e isolamento de dados por armazém	Alta
RF4	Gestão de prateleiras, stock e ordens de picking	Alta
RF5	Coordenação lógica operador-robô com ponto de encontro	Alta
RF6	Cálculo de rotas por algoritmo exato e métricas de eficiência	Alta
RF7	Guiar o operador (app) até à prateleira e ao ponto de encontro	Alta
RF8	Catálogo de robôs com recomendação automática por layout (SAD)	Alta
RF9	Decisão multicritério humano/robô (peso, fragilidade, velocidades distintas)	Alta
RF10	Três algoritmos de planeamento (exato, guloso, meta-heurística) com comparação	Alta
RF11	Validação física de encaixe (artigo vs. prateleira) e de acessibilidade das prateleiras	Alta

Tabela 3 - Requisitos funcionais e respetiva prioridade.

A Figura 9 apresenta o diagrama de casos de uso do sistema, cobrindo os requisitos funcionais RF1 a RF11: o Gestor configura o armazém, gere stock e prateleiras e consulta a recomendação de robô e o planeamento; o Operador regista-se, consulta as suas tarefas e confirma cada recolha. O ator Operador é duplicado do lado direito do diagrama apenas para evitar o cruzamento de linhas com os casos de uso partilhados (autenticação e confirmação de recolha).

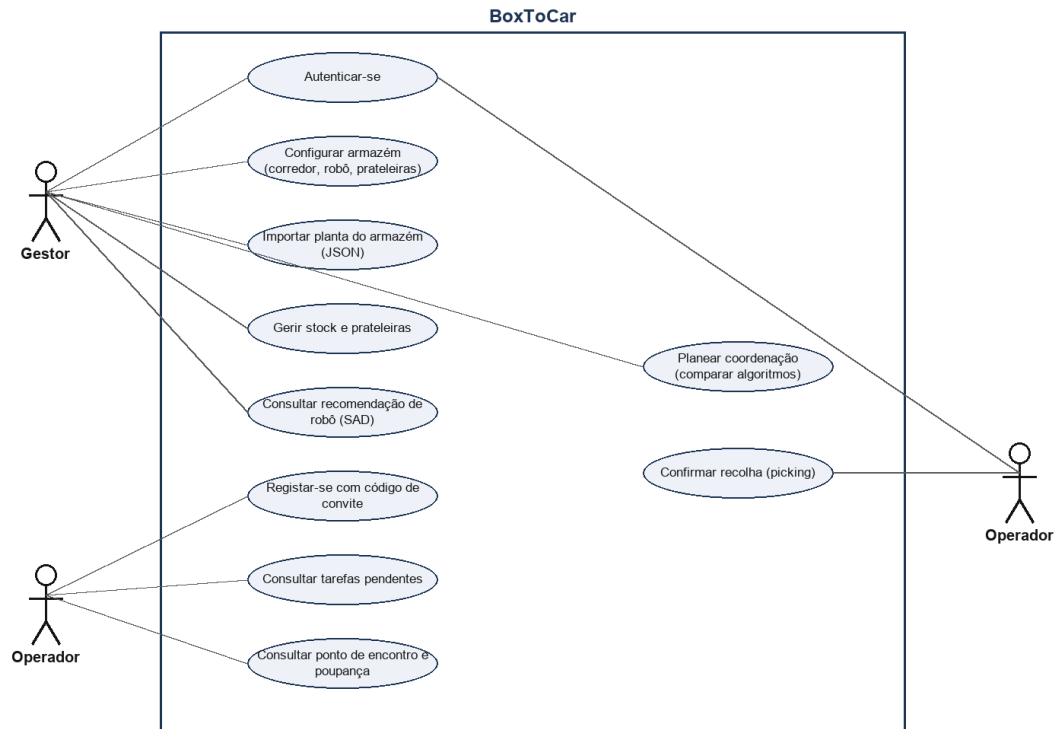


Figura 1 - Diagrama de casos de uso.

6.1.1. Requisitos não funcionais

- Segurança - palavras-passe cifradas com bcrypt (fator de custo 12, nunca em texto simples); toda a rota que altera dados exige um token de sessão válido; e cada consulta é filtrada pelo ArmazémID do utilizador autenticado, impedindo o acesso a dados de outro armazém.
- Usabilidade - a partir do início de sessão, o operador chega ao ponto de encontro e à poupança de esforço de uma tarefa em, no máximo, dois passos (ecrã de tarefas do armazém e toque na tarefa); o backoffice web e a aplicação móvel partilham a mesma paleta de cores e terminologia.
- Portabilidade - a aplicação do operador mantém uma única base de código Flutter (app_operador/lib), compilável sem alterações para pelo menos três alvos (Android, iOS e Web).
- Desempenho - para um lote de até 6 recolhas pendentes, a heurística gulosa e a meta-heurística devolvem uma solução em menos de 50 ms (3 ms e 38 ms, respetivamente, medidos no capítulo 13); o algoritmo exato, de complexidade fatorial, está limitado a um máximo de 7 tarefas por lote para manter o tempo de cálculo abaixo de 1 segundo (895 ms medidos com 6 tarefas).
- Manutenibilidade - a lógica de planeamento e de decisão (algoritmos.js, coordenacao.js, custoDecisao.js e a parte de recomendação de robos.js) está isolada em funções puras, sem I/O de rede nem de base de dados, o que permite testá-las unitariamente invocando-as diretamente sobre estruturas de dados em memória (secção 13.4).

6.2. Arquitetura da solução

O sistema é constituído por três componentes (Figura 1). O backend (Node.js/Express) constitui o núcleo do sistema: expõe a API REST, comunica com a base de dados e concentra a lógica de coordenação. O backoffice web (HTML, CSS e JavaScript), servido pelo backend, disponibiliza o mapa interativo e a visualização da coordenação. A aplicação móvel (Flutter) guia o operador. Os clientes comunicam com a mesma API, partilhando dados e regras de negócio.

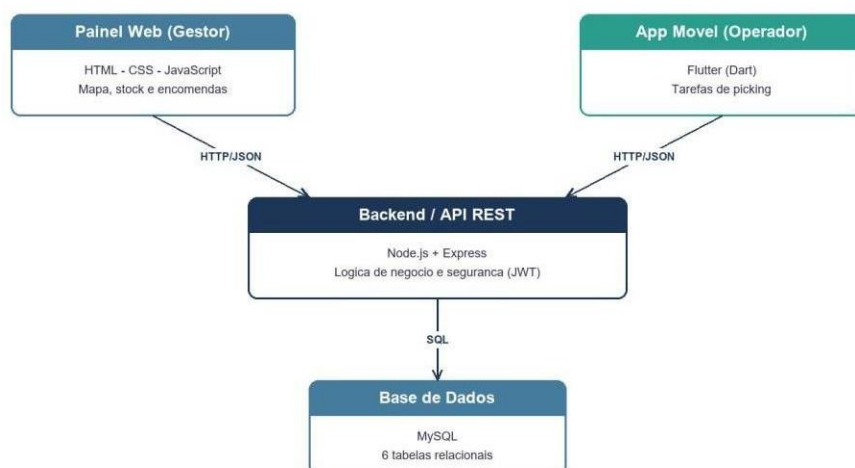


Figura 2 - Arquitetura cliente-servidor da solução.

6.3. Modelo de dados

A base de dados é composta por oito tabelas principais (Figura 2 e Tabela 3). Cada produto corresponde a uma unidade física de stock, o que permite calcular o nível de stock de um artigo contando os registos correspondentes.

Figura — Modelo de dados (Diagrama Entidade-Relacao)

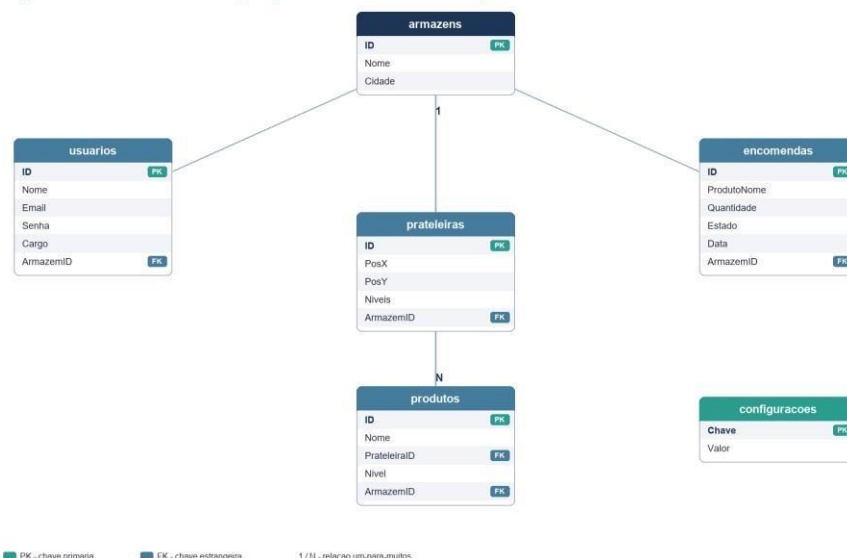


Figura 3 - Modelo de dados (diagrama entidade-relação).

Tabela	Descrição
armazens	Cada armazém: nome, cidade, layout físico (largura do corredor, tamanho único das prateleiras) e robô fixado (opcional).
usuarios	Contas de gestor/operador e armazém associado.
prateleiras	Posição (X,Y) e níveis de cada estrutura.
produtos	Cada registo é uma unidade física de stock; liga-se ao artigo por ArtigoID (chave estrangeira) e à prateleira por PrateleiraID.
Tabela	Descrição
encomendas	Ordens de picking e respetivo estado; liga-se ao artigo por ArtigoID (chave estrangeira), já não pelo nome.
configuracoes	Parâmetros do armazém (dimensões da grelha).
artigos	Dados mestre do SKU: nome, peso, dimensões e fragilidade - usados na decisão multicritério.
tiposrobo	Catálogo de robôs: capacidade de carga, velocidade, tipo de navegação, largura mínima de corredor e dimensões físicas (comprimento, largura, altura).

Tabela 4 - Tabelas principais da base de dados.

O modelo evoluiu de uma primeira versão em que as encomendas referenciavam o produto apenas pelo nome para o esquema atual, em que tanto os produtos como as encomendas se ligam ao artigo por uma chave estrangeira (ArtigoID), com restrições de integridade referencial reais impostas pelo SGBD - uma migração de esquema em código (não apenas cliques no phpMyAdmin), que revelou e corrigiu registos órfãos de armazéns já apagados, sintoma direto da ausência de FK na versão anterior. Mantêm-se, ainda assim, duas simplificações conscientes, próprias de uma prova de conceito: cada unidade de stock é um registo distinto na tabela produtos, em vez de uma coluna de quantidade, o que facilita a leitura e a demonstração mas não escala para grandes volumes; e a tabela configuracoes assume, no protótipo, um único armazém por gestor. Estas opções são retomadas nas limitações (secção 15.4).

6.4. Validação física de encaixe e de acessibilidade

O sistema impõe duas validações adicionais, para que o modelo do armazém se mantenha fisicamente coerente. Primeiro, a entrada de stock verifica se o artigo cabe na prateleira: compara as suas três dimensões com o tamanho único das prateleiras do armazém, testando as duas orientações possíveis no plano horizontal, e recusa a operação se não couber em nenhuma delas ou se exceder a altura disponível por nível. Segundo, a construção de uma prateleira nova - e a importação de uma planta completa - é recusada se deixar alguma prateleira, nova ou já existente, sem nenhuma célula livre à sua volta: sem uma célula de acesso, nem o operador nem o robô teriam onde se colocar para arrumar ou recolher um artigo ali, o que tornaria essa prateleira inutilizável assim que ficasse cercada. Esta segunda verificação corrige uma inconsistência identificada durante os testes: era possível encaixar prateleiras umas nas

outras ao ponto de bloquear o acesso a uma delas depois de já ter stock guardado, impedindo depois a sua recolha.

6.5. Planeamento e metodologia

O desenvolvimento seguiu o modelo de desenvolvimento incremental (Sommerville, 2016), no qual o sistema é construído por uma sucessão de incrementos, cada um acrescentando um conjunto reduzido de funcionalidades a uma versão já operacional. Esta abordagem insere-se na tradição do desenvolvimento iterativo e incremental, cuja utilização em engenharia de software é anterior aos métodos ágeis (Larman & Basili, 2003).

A escolha deste modelo, em vez de um processo sequencial, deveu-se ao facto de os requisitos terem sido clarificados à medida que o sistema tomava forma - em particular o algoritmo de coordenação, cujo critério de escolha do ponto de encontro só estabilizou após as primeiras experiências sobre a grelha. Optou-se por não adotar um método ágil de equipa, como o Scrum, por o projeto ser individual: as práticas de coordenação entre elementos não seriam aplicáveis.

Cada incremento seguiu o mesmo ciclo: (1) seleção da funcionalidade a partir dos requisitos funcionais RF1–RF10; (2) análise e desenho, incluindo as alterações necessárias ao modelo de dados; (3) implementação no backend e, em seguida, nos clientes; (4) verificação contra os critérios de aceitação definidos no capítulo 13; e (5) integração no sistema existente. O código foi versionado com Git, com um commit descritivo por funcionalidade ou correção, pelo que o histórico do repositório documenta a sequência de incrementos e assegura a rastreabilidade das alterações.

Reconhece-se que a aplicação do modelo foi informal: não houve iterações de duração fixa nem um registo formal de incrementos além do histórico de commits.

7. Manual de Utilizador

7.1. Introdução

Este manual descreve, passo a passo, como utilizar o BoxToCar. Destina-se a qualquer utilizador e não exige conhecimentos técnicos. O sistema tem dois perfis: o Gestor, que administra o armazém através do backoffice web, e o Operador, que executa as recolhas através da aplicação móvel. As figuras referidas ao longo do manual correspondem às apresentadas nos capítulos anteriores do relatório.

7.2. Guia do Gestor (Backoffice Web)

7.2.1. Criar conta e armazém

1. No ecrã de acesso, clique em "Regista-te aqui".
2. Preencha o seu nome, email e palavra-passe.
3. Indique o nome do armazém (e, opcionalmente, a cidade) e clique em "CRIAR CONTA". O armazém é criado e associado à sua conta de Gestor.

7.2.2. Iniciar sessão

4. No ecrã de acesso (Figura 3), introduza o email e a palavra-passe e clique em "ENTRAR NO SISTEMA".

7.2.3. O painel e o mapa do armazém

O painel (Figura 4) apresenta ao centro o mapa em grelha: cada quadrado é uma posição, as prateleiras estão a azul e o robô ($\square$) desloca-se sobre o mapa. As ferramentas de gestão estão à direita e por baixo.

7.2.4. Definir o tamanho e construir prateleiras

5. Em "Expandir Chão", defina o número de colunas (Max X) e de linhas (Max Y).
6. Em "Construir Prateleira", indique a posição (X, Y) e os níveis e clique em "CRIAR ESTRUTURA".

Nota: Em alternativa, pode carregar todo o layout de uma só vez importando uma planta em ficheiro JSON, no cartão "Importar Planta".

Nota: o sistema recusa construir (ou importar) uma prateleira que deixe outra sem nenhum acesso livre à volta - o erro indica quais as prateleiras em causa, para o layout ser corrigido antes de voltar a tentar.

7.2.5. Configurar o armazém

7. No painel "Configuração do Armazém", defina a largura do corredor (m).
8. Defina o tamanho único das prateleiras deste armazém: largura, profundidade e altura por nível, em cm.

9. Escolha, opcionalmente, um robô fixo na lista (em vez de "Automático") para sobrepor a recomendação do sistema.
10. Clique em "GUARDAR CONFIGURAÇÃO".

7.2.6. Dar entrada de produtos

11. Em "Entrada de Produto", escreva o nome/SKU e a quantidade.
12. Indique a posição da prateleira (X, Y) e o nível e clique em "ARRUMAR NO STOCK" (Figura 5). A prateleira passa a mostrar o número de artigos (□).
13. Indique também o peso, as dimensões e se o artigo é frágil. O sistema recusa a entrada se o artigo não couber na prateleira (mesmo rodado) ou exceder a altura do nível.

7.2.7. Gerar ordens de picking

14. Em "Simulador ERP", escreva o produto e a quantidade e clique em "+" para o adicionar ao carrinho (pode remover no "X").
15. Clique em "GERAR ORDEM DE PICKING". O sistema valida o stock e cria as tarefas.

Nota: Se tentar encomendar mais do que existe (contando com as encomendas pendentes), a operação é bloqueada.

7.2.8. Planear a coordenação humano-máquina

16. Clique numa prateleira com produto e, no detalhe, clique em "PLANEAR COORDENAÇÃO".
 17. No mapa surgem o ponto de encontro (M), o depósito (D), a rota do robô (vermelho) e a do operador (verde), e um painel com as métricas de poupança (Figura 7).
- Nota:** A visualização mantém-se até fechar no "X" do painel ou até realizar uma nova ação (gerar ordem, criar prateleira, etc.).

7.3. Guia do Operador (Aplicação Móvel)

7.3.1. Criar conta e iniciar sessão

18. No ecrã de início de sessão, toque em "Não tens conta? Cria uma aqui".
19. Preencha os dados, escolha o armazém na lista e toque em "CRIAR CONTA".
20. Indique também o código de convite fornecido pelo gestor do armazém - sem ele, o registo é recusado.
21. Para entrar, introduza o email e a palavra-passe e toque em "ENTRAR".

Nota: A aplicação mantém a sessão iniciada entre utilizações; só terá de entrar de novo se terminar sessão ou se esta expirar.

7.3.2. Executar uma recolha

- 22.** Consulte a lista de tarefas pendentes; cada uma indica o produto, o peso, se é frágil, e a localização (Figura 8).
- 23.** Numa tarefa, toque em "PONTO DE ENCONTRO (ROBÔ)" para ver a decisão do sistema - se a recolha é feita sozinho ou com o apoio do robô, e porquê -, o ponto de encontro (quando aplicável) e a poupança de esforço.
- 24.** Se a prateleira não tiver nenhuma célula livre à volta, a aplicação avisa que não é possível calcular a coordenação com o robô; isso não impede confirmar a recolha manualmente, sem o robô.
- 25.** Recolha o artigo na prateleira, leve-o até ao ponto de encontro e entregue-o ao robô.
- 26.** Toque em "CONFIRMAR RECOLHA". A tarefa é concluída e o stock é atualizado.

7.4. Resolução de Problemas (FAQ)

Não consigo iniciar sessão. Verifique o email e a palavra-passe e confirme que o servidor está ligado.

A minha sessão expirou. Por segurança, a sessão tem duração limitada; basta iniciar sessão novamente.

Não consigo encomendar um produto. O sistema impede encomendar mais unidades do que as disponíveis; verifique o stock ou dê entrada de mais unidades.

A coordenação não aparece no mapa. Confirme que clicou em "PLANEAR COORDENAÇÃO" a partir de uma prateleira e, se necessário, recarregue a página (Ctrl+Shift+R).

Não consigo criar conta na aplicação do operador. Confirme que introduziu o código de convite fornecido pelo gestor do seu armazém; sem ele, o registo de operador é recusado.

Aparece "a prateleira está bloqueada" ao tentar ver o ponto de encontro. Significa que essa prateleira não tem nenhuma célula livre à volta, pelo que o robô não consegue coordenar-se ali. A recolha continua a poder ser confirmada normalmente, sem o apoio do robô.

8. Autenticação, Autorização e Multi-armazém

O início de sessão valida as credenciais (comparando a palavra-passe com o hash bcrypt) e emite um token JWT com validade de oito horas, que incorpora a identidade, o papel e o armazém do utilizador (Figura 3). Um middleware valida o token em todas as rotas sensíveis e disponibiliza os dados do utilizador; o armazém é sempre obtido do token, e nunca de parâmetros do cliente, garantindo o isolamento de dados. Foram implementados dois papéis: Gestor (backoffice web) e Operador (aplicação móvel).

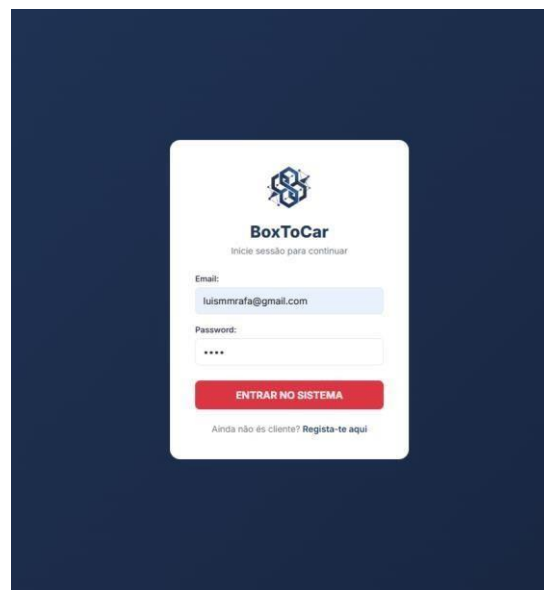


Figura 4 - Ecrã de início de sessão do backoffice web.

Numa primeira versão, o armazém era indicado pelo cliente, o que permitiria, em teoria, aceder a dados de outro armazém; a correção passou por derivar o armazém exclusivamente do token e por validar a posse dos recursos nas operações de remoção e conclusão. Os testes (capítulo 13) confirmaram que um utilizador não acede nem altera dados de outro armazém e que as rotas sensíveis rejeitam pedidos sem token válido.

9. Gestão do Armazém e Mapa Interativo

O backoffice apresenta o armazém como uma grelha 2D, em que cada célula é uma posição (Figura 4). As prateleiras são assinaladas e, ao serem selecionadas, mostram os produtos que contêm. As dimensões da grelha são configuráveis e ajustam-se à distribuição das prateleiras.



Figura 5 - Backoffice web com o mapa interativo do armazém.

As prateleiras podem ser criadas individualmente ou importadas em conjunto a partir de um ficheiro de planta em formato JSON (RF2), o que concretiza a importação dinâmica de layouts prevista na proposta. A importação substitui o layout anterior numa transação, evitando sobreposições. Esta grelha constitui o espaço sobre o qual operam os algoritmos de rota e de coordenação descritos no capítulo 11.

10. Gestão de Stock e Ordens de Picking

O gestor dá entrada de produtos indicando o nome, a posição e a quantidade; cada unidade é um registo, e o nível de stock corresponde ao número de registos. As ordens de picking são geradas por um simulador que reproduz a chegada de encomendas de um sistema ERP (Figura 5).

A interface é dividida em duas secções principais:

- Entrada de Produto:** Possui campos para 'Nome / SKU:' (contendo 'monitor') e 'Qtd:' (contendo '2'). Abaixo, há campos para 'Ir ao X:' (5), 'Ir ao Y:' (2) e 'Nível:' (1). Um botão verde 'ARRUMAR NO STOCK' está na base.
- Simulador ERP (Múltiplos Itens):** Possui um campo 'Produto' e um botão '+' para adicionar itens. Abaixo, há uma lista de itens com checkmarks e botões de remoção: '1x monitor' e '2x rato'. Um botão vermelho 'GERAR ORDEM DE PICKING' está na base.

Figura 6 - Entrada de produtos e simulador de ordens de picking (ERP).

Antes de criar cada ordem, o sistema valida a disponibilidade, calculando o stock disponível como a diferença entre o stock físico e as quantidades já comprometidas em encomendas pendentes, o que impede a sobre-alocação. Ao concluir uma tarefa, o operador desencadeia a baixa de stock; todas as operações que envolvem várias alterações ocorrem dentro de transações. A Figura 6 resume o fluxo do processo.

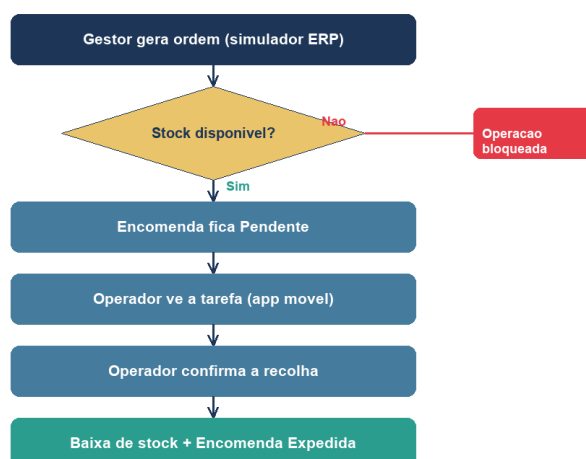


Figura 7 - Fluxo do processo de picking.

11. Coordenação Humano-Máquina

Este capítulo descreve o núcleo do projeto e a principal contribuição face a um WMS convencional: a coordenação lógica entre o operador humano e o robô virtual, que concretiza o paradigma de picking colaborativo da Indústria 5.0.

11.1. Modelo de coordenação

O armazém é modelado como uma grelha 2D em que as prateleiras são obstáculos e as restantes células são corredores livres. O robô circula exclusivamente pelo corredor principal (o perímetro da grelha), partindo de um ponto de expedição (o depósito). Para cada recolha, o operador dirige-se à prateleira, recolhe o artigo e transporta-o apenas até ao ponto de encontro - a célula do corredor principal mais próxima da prateleira -, onde o entrega ao robô. O robô assume então o transporte pesado até à expedição, libertando o operador para a tarefa seguinte.

O robô que participa nesta coordenação não é único nem fixo: é escolhido - pelo gestor, ou recomendado automaticamente - de entre um catálogo de tipos com especificações reais (capacidade de carga, velocidade, tipo de navegação), descrito na secção seguinte. Se o artigo exceder a capacidade do robô selecionado, a opção de o usar é fisicamente inviável, e não apenas desvantajosa; essa e outras condições formam a decisão multicritério detalhada mais adiante neste capítulo.

11.2. Recomendação de robô (Sistema de Apoio à Decisão)

O módulo `robos.js` mantém o catálogo de tipos de robô e implementa a recomendação: dado o layout de um armazém (em concreto, a largura do seu corredor principal) e o perfil de carga das recolhas pendentes, filtra os tipos fisicamente compatíveis - os que cabem no corredor e aguentam o artigo mais pesado pendente - e pontua-os pelo equilíbrio entre o aproveitamento da capacidade de carga e a velocidade, penalizando tanto a subcapacidade (inviável) como o sobredimensionamento (desperdício de investimento). O robô com maior pontuação é recomendado; o gestor pode ainda fixar manualmente o robô de um armazém, sobrepondo-se à recomendação automática. Este módulo é o que dá ao sistema o comportamento de um SAD, referido no capítulo 5: a aplicação propõe, para cada armazém, o robô mais adequado ao seu layout, em vez de assumir um único modelo fixo e arbitrário.

Estes dois parâmetros do armazém - a largura do corredor e, opcionalmente, o robô fixado manualmente - são geridos pelo próprio gestor num painel de configuração dedicado do backoffice, juntamente com o tamanho único das prateleiras usado na validação de encaixe (secção 6.4). Sem esta configuração, a recomendação ficaria sempre presa a um valor por omissão desligado da realidade física de cada armazém.

11.3. Cálculo do ponto de encontro

O ponto de encontro é determinado de modo a minimizar a distância percorrida pelo operador com carga. Para tal, executa-se uma procura em largura (BFS) multi-origem a partir das células de acesso à prateleira (os seus vizinhos livres), obtendo-se a distância de qualquer célula livre até à recolha. O ponto de encontro é a célula do corredor principal com menor distância assim obtida. A rota do robô resulta de uma segunda BFS, a partir do depósito, e a rota do operador com carga da reconstrução do caminho até ao ponto de encontro.

Este cálculo geométrico - ponto de encontro, rotas e distâncias - não decide, por si só, se o robô deve ser usado: fornece apenas os valores de distância de que a decisão multicritério e os três algoritmos de planeamento (descritos a seguir) precisam para escolher a atribuição e a sequência de menor custo.

11.4. Decisão multicritério humano-robô

Para cada recolha, o módulo `custoDecisao.js` compara duas opções - humano sozinho até ao depósito, ou humano até ao ponto de encontro seguido do robô - e escolhe a de menor custo, em vez de decidir apenas pela distância. O custo de cada opção é o tempo de deslocação de cada agente à sua própria velocidade (o operador a pé; o robô à velocidade do tipo selecionado), ponderado para dar mais peso ao esforço do operador do que ao tempo do robô (70% / 30%, na configuração atual, refletindo que poupar tempo ao operador vale mais do que poupar tempo ao robô). Duas condições adicionais afastam a decisão da distância pura: um artigo cujo peso exceda a capacidade do robô selecionado torna essa opção inviável, não apenas pior, obrigando o operador a seguir sozinho; e um artigo frágil recebe uma penalização de 35% no tempo do robô, refletindo o risco acrescido de manuseamento na entrega no ponto de encontro. A opção escolhida, os tempos de cada agente e os alertas gerados (peso excedido, frágil) são devolvidos junto com o plano de coordenação, para o gestor e o operador perceberem a razão da decisão, e não apenas o resultado.

11.5. Os três algoritmos de planeamento

O problema a resolver deixou de ser só geométrico: para um conjunto de recolhas pendentes, é preciso decidir, para cada uma, se vai por humano sozinho ou por humano com o apoio do robô, e a ordem pela qual o operador as executa, minimizando o custo total (esforço do operador, tempo do robô e penalizações de peso/frágilidade). A posição em que o operador fica no final de uma recolha - o ponto de encontro, se usou o robô, ou o depósito, se não usou - condiciona a distância até à recolha seguinte, pelo que a atribuição e a sequência não podem ser otimizadas em separado: o espaço de procura cresce em $N! \times 2^N$ (ordens possíveis vezes atribuições possíveis por tarefa), o que torna a busca exaustiva impraticável além de poucas recolhas. É esta a razão pela qual se comparam três algoritmos.

Exato (força bruta). Testa todas as ordens e todas as combinações de atribuição fisicamente viáveis (excluindo à partida as que excedem a capacidade do robô selecionado) e devolve a de menor custo. Serve de referência para avaliar as duas heurísticas seguintes, mas só é viável para poucas recolhas - a implementação recusa-se a correr acima de 7 tarefas.

Heurística gulosa. Constrói a solução passo a passo: em cada passo escolhe a recolha pendente mais próxima da posição atual do operador e decide, na hora, se compensa usar o robô ou seguir sozinho, sem olhar para trás. É rápida (ordem de milissegundos, mesmo com dezenas de recolhas) mas não garante o ótimo.

Meta-heurística. Parte da solução gulosa e explora vizinhanças, trocar a ordem de duas recolhas, ou inverter a atribuição de uma aceitando por vezes soluções piores, com probabilidade decrescentes ao longo das iterações, para escapar a mínimos locais que a heurística gulosa não consegue evitar. Na prática, aproxima-se do ótimo em muito menos tempo do que o algoritmo exato.

O pseudocódigo simplificado dos três algoritmos (a implementação completa consta do Apêndice C):

```
função resolverExato(tarefas, contexto):
  para cada ordem em permutações(tarefas):
    para cada atribuição em combinações_viáveis(tarefas):
      custo <- avaliarSequencia(ordem, atribuição, contexto)
      guarda o melhor (ordem, atribuição, custo)
  devolve o melhor

função resolverGuloso(tarefas, contexto):
  posição <- depósito
  enquanto houver tarefas por atribuir:
    próxima <- tarefa pendente mais próxima de posição
    atribuição[próxima] <- opção de menor custo (humano só | humano+robô)
    posição <- ponto de encontro ou depósito, conforme a atribuição
  devolve (ordem, atribuição)

função resolverMetaheurística(tarefas, contexto):
  solução <- resolverGuloso(tarefas, contexto)
  temperatura <- T0
  repetir N vezes:
    vizinho <- trocar ordem(2 tarefas) OU inverter atribuição(1 tarefa)
    Δ <- custo(vizinho) - custo(solução)
    se Δ < 0 ou aleatório() < exp(-Δ / temperatura): solução <- vizinho
    temperatura <- temperatura × arrefecimento
  devolve a melhor solução encontrada
```

A Tabela 4 ilustra os três algoritmos sobre um armazém de referência (7×6, seis recolhas pendentes, robô AMR Ligeiro - 15 kg, 1,2 m/s): o algoritmo exato encontra o custo ótimo, a heurística gulosa calcula quase instantaneamente, mas fica 23% acima do ótimo, e a meta-heurística aproxima-se do ótimo (a menos de 8%) num tempo ainda muito inferior ao exato. Uma das seis recolhas (um artigo de 22 kg) excede a capacidade do robô selecionado e é corretamente atribuída ao operador sozinho pelos três algoritmos, ilustrando a restrição de viabilidade descrita na secção anterior.

Algoritmo	Custo total (s)	Tempo de cálculo	Desvio ao ótimo	Nº recolhas	Observação
Exato	25,77	895 ms	0%	6	Ótimo; impraticável para muitas recolhas
Guloso	31,78	3 ms	23,3%	6	Mais rápido; qualidade mais baixa
Meta-heurística	27,77	38 ms	7,8%	6	Perto do ótimo, muito mais rápida que o exato

Tabela 5 - Comparação dos três algoritmos de planejamento (armazém 7×6, 6 recolhas).

11.6. Implementação

A lógica foi isolada em módulos próprios do backend, independentes da base de dados e, por isso, testáveis de forma unitária: `coordenacao.js` calcula distâncias e rotas sobre a grelha (BFS); `robos.js` mantém o catálogo de robôs e a recomendação por layout; `custoDecisao.js` decide a atribuição de cada recolha; e `algoritmos.js` implementa os três algoritmos de planejamento sobre estas peças. Um endpoint da API recebe a prateleira-alvo (por posição ou por produto) para o plano de uma única recolha, e outro recebe o conjunto de recolhas pendentes de um armazém para o planejamento em lote, devolvendo os resultados dos três algoritmos lado a lado.

```
for (const [cel, d] of distAcesso)
  if (ehCorredorPrincipal(cel) && d < melhor) {
    melhor = d; pontoEncontro = cel;
  }
```

11.7. Visualização

O backoffice web permite visualizar, sobre o mapa, o ponto de encontro, o depósito e as rotas do robô e do operador, acompanhados das métricas de eficiência (Figura 7). A aplicação móvel apresenta ao operador o ponto de encontro e a poupança de esforço correspondente (Figura 8).

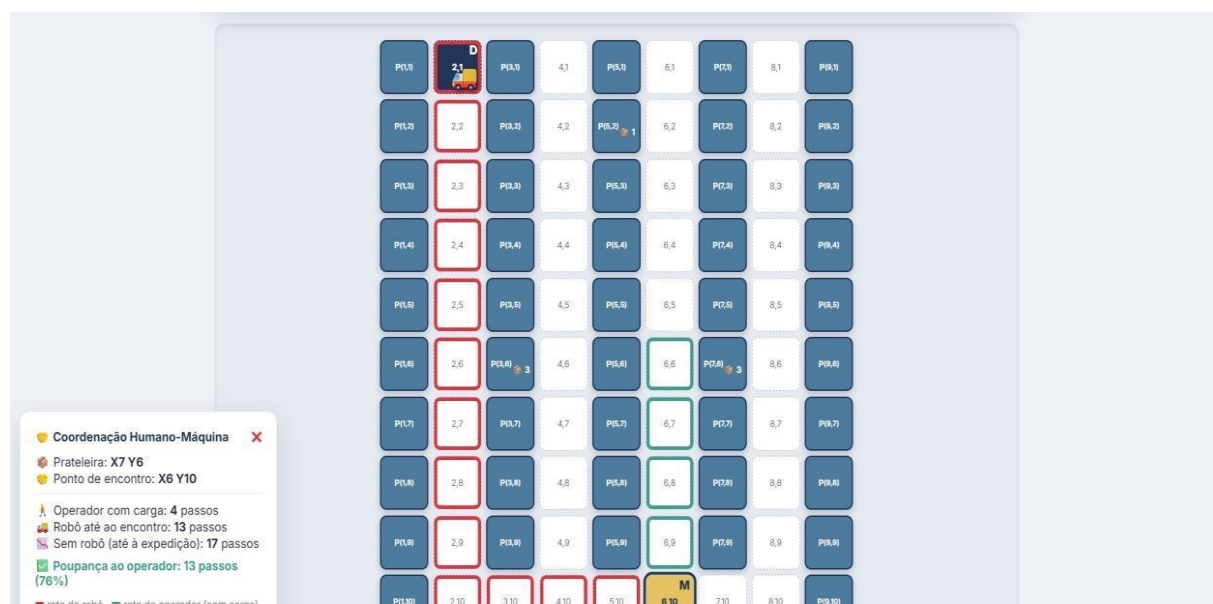


Figura 8 - Coordenação humano-máquina: ponto de encontro e rotas calculadas.

11.8. Métricas de eficiência

Para cada recolha, o sistema quantifica três grandezas: a distância percorrida pelo operador com carga (até ao ponto de encontro), a distância percorrida pelo robô (do depósito ao ponto de encontro) e a poupança - a distância que o operador percorreria com carga caso tivesse de transportar o artigo até à expedição sem o apoio do robô. Estas métricas, além de tornarem visível o valor da coordenação, servem de base à validação quantitativa apresentada no capítulo 13.

A estas métricas geométricas juntam-se agora duas leituras adicionais: o tempo estimado de cada agente, calculado a partir da velocidade do tipo de robô selecionado e de uma velocidade de referência para o operador, e a própria atribuição decidida (humano sozinho ou humano com o apoio do robô), com o motivo que a justifica. A poupança deixa, assim, de ser expressa apenas em passos, passando a refletir também o tempo e as restrições físicas de cada recolha.

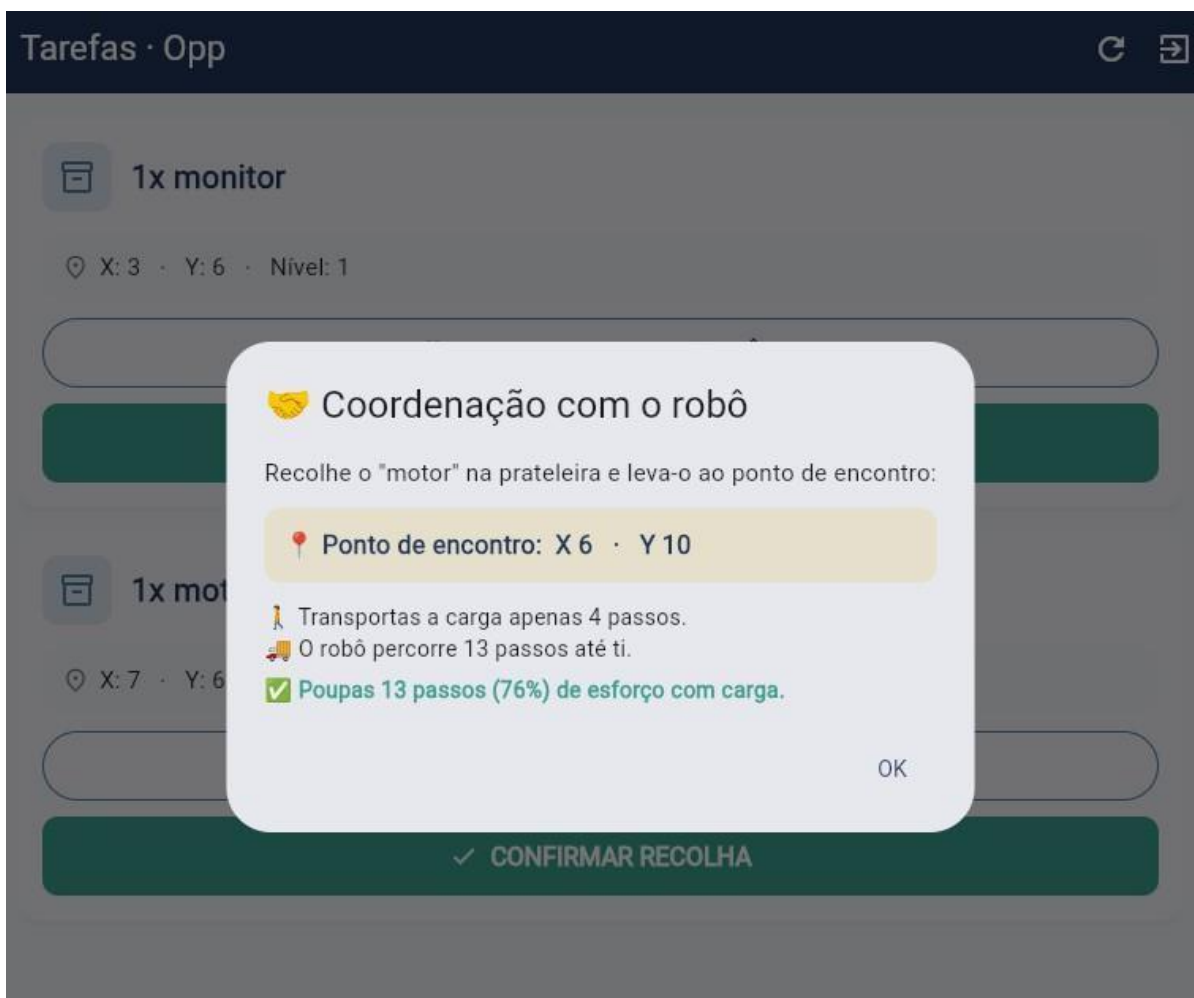


Figura 9 - Aplicação móvel do operador: tarefa e ponto de encontro com o robô

12. Aplicação Móvel do Operador

A aplicação móvel, desenvolvida em Flutter, permite ao operador iniciar sessão, consultar as tarefas pendentes do seu armazém e, para cada uma, consultar o ponto de encontro com o robô e a poupança de esforço associada (Figura 8), antes de confirmar a recolha e desencadear a baixa de stock. O token de autenticação é guardado no dispositivo, mantendo a sessão ativa entre utilizações.

A Figura 9 detalha, num diagrama de sequência, o fluxo completo de uma recolha coordenada: desde a consulta das tarefas pendentes pelo operador, passando pelo cálculo do ponto de encontro e pela decisão multicritério de atribuição, até à confirmação da recolha e à atualização do stock - evidenciando a interação entre a aplicação móvel, a API e os módulos de coordenação e decisão.

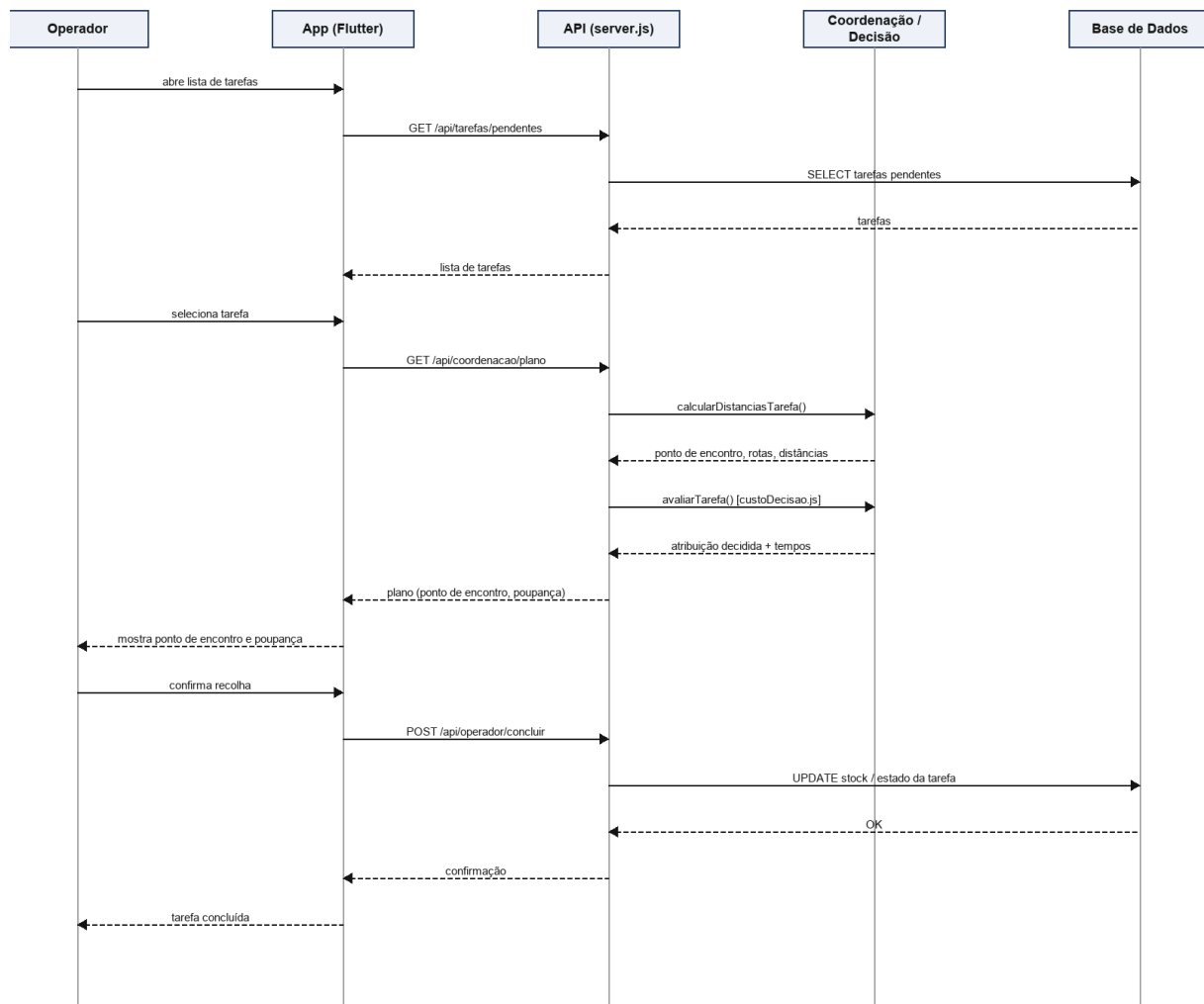


Figura 10 - Diagrama de sequência

A comunicação com a API é feita por pedidos HTTP com o token no cabeçalho de autorização; perante uma sessão expirada, a aplicação limpa o token e regressa ao início de sessão. A interface partilha a identidade visual do backoffice, garantindo consistência entre as duas vertentes do sistema

13. Testes de Validação

13.1. Metodologia de teste

A validação seguiu uma abordagem de caixa-preta, exercitando o sistema através da sua API REST, de forma independente das interfaces gráficas. Os testes foram executados num ambiente controlado (Node.js com base de dados MySQL local), sobre um armazém de referência de dimensão 7×6, com corredores definidos, e recorrendo a um cliente automatizado desenvolvido em Python (biblioteca urllib), que emite os pedidos e verifica automaticamente as respostas. Distinguiram-se dois grupos de testes: testes funcionais, que verificam o comportamento das operações críticas face a resultados esperados; e testes ao algoritmo de coordenação, que medem quantitativamente a eficiência dos pontos de encontro calculados. O critério de aceitação foi, para os primeiros, a conformidade com o resultado esperado e, para os segundos, a obtenção de rotas válidas e de poupança não negativa em todos os cenários.

13.2. Testes funcionais

A Tabela 5 sintetiza os principais casos de teste funcionais, com o requisito funcional que cada um valida, o objetivo, a ação, o resultado esperado e o resultado obtido. Cobre-se assim explicitamente cada requisito funcional de RF2 a RF11 - RF1 (arquitetura em três componentes) não corresponde a um único caso de teste, sendo antes validado holisticamente pela execução bem-sucedida de todos os restantes, que exercitam a API, o backoffice e a aplicação móvel em conjunto.

ID	Requisito	Objetivo / ação	Resultado esperado	Obtido
T1	RF3	Aceder a rota sensível sem token	Rejeição (HTTP 401)	Conforme
T2	RF3	Aceder a dados de outro armazém	Acesso negado / isolado	Conforme
T3	RF4	Encomendar acima do stock disponível	Operação bloqueada (400)	Conforme
T4	RF4, RF7	Concluir tarefa de picking	Baixa de stock e estado atualizado	Conforme
T5	RF2	Importar nova planta (JSON)	Layout anterior substituído	Conforme
T6	RF5, RF6	Plano de coordenação de produto inexistente	Erro tratado (404)	Conforme

T7	RF8	Pedir recomendação de robô para um corredor estreito - GET /api/robos/recomendar com corredor mais estreito que a largura mínima de alguns tipos de robô	Só são recomendados robôs cujo corredor mínimo cabe na largura configurada	Conforme
T8	RF9	Planejar recolha de artigo acima da capacidade do robô - GET /api/coordenacao/plano para um artigo cujo peso excede a capacidade do robô do armazém	Atribuição forçada a operador sozinho (robô excluído por peso)	Conforme
T9	RF10	Comparar os três algoritmos sobre o mesmo lote de tarefas - GET /api/planeamento/otimizar com as recolhas pendentes de um armazém	Exato, guloso e meta-heurística devolvem soluções válidas; o exato é ótimo ou empata com os outros	Conforme
T10	RF11	Guardar artigo com dimensões maiores que a prateleira - POST /api/produtos/novo com dimensões que não cabem em nenhuma orientação da prateleira	Operação recusada (400)	Conforme
T11	RF11	Criar prateleira que bloqueia o acesso a outra já existente - POST /api/prateleiras/nova numa posição que rodeia uma prateleira sem deixar célula livre	Operação recusada, prateleira não criada	Conforme

Tabela 6 - Casos de teste funcionais e resultados obtidos.

13.3. Testes ao algoritmo de coordenação

Para avaliar quantitativamente a coordenação, calcularam-se os planos de recolha para um conjunto de prateleiras representativas do armazém de referência, registando-se a distância do operador com carga, a distância do robô e a poupança de esforço (Tabela 6). A poupança percentual traduz a fração da distância com carga que é evitada ao operador graças ao robô.

Prateleira	Ponto de encontro	Operador c/ carga	Robô	Poupança	Poupança %
(5,3)	(6,1)	2 passos	9 passos	3 passos	60%
(5,5)	(6,6)	1 passo	8 passos	4 passos	80%
(4,5)	(3,6)	1 passo	5 passos	3 passos	75%
(2,3)	(1,4)	1 passo	5 passos	1 passo	50%
(5,6)	(6,6)	0 passos	8 passos	8 passos	100%

Tabela 7 - Resultados do algoritmo de coordenação (armazém 7×6).

Para verificar que o algoritmo generaliza a diferentes plantas, repetiu-se a avaliação num segundo layout, de dimensão 5×7 e com uma disposição distinta das prateleiras e um corredor transversal (Tabela 7). Os resultados mantêm quase o mesmo padrão, com poupanças entre 80% e 100%, confirmando que o comportamento não depende do layout específico usado na Tabela 6.

Prateleira	Ponto de encontro	Operador c/ carga	Robô	Poupança	Poupança %
(5,3)	(5,4)	0 passos	6 passos	6 passos	100%
(1,6)	(2,7)	1 passo	6 passos	4 passos	80%
(3,7)	(2,7)	0 passos	6 passos	6 passos	100%

Tabela 8 - Resultados do algoritmo de coordenação num segundo layout (5×7).

13.4. Testes unitários ao módulo de coordenação

Como o módulo de coordenação (coordenacao.js) é independente da base de dados, foi também testado de forma unitária, invocando diretamente a função de planeamento sobre grelhas construídas em código. Para cada cenário, verificou-se que o ponto de encontro pertence ao corredor principal, que as rotas do operador e do robô são contíguas e não atravessam prateleiras, e que a poupança calculada é coerente com as distâncias. Estes testes unitários complementam os testes de integração pela API (secções 13.2 e 13.3), cobrindo isoladamente a lógica do algoritmo.

13.5. Testes comparativos aos três algoritmos de planeamento

Para além dos testes ao ponto de encontro de uma única recolha (secção 13.3), os três algoritmos de planeamento foram comparados sobre um conjunto de recolhas pendentes, em dois layouts distintos, registando o custo total da solução e o tempo de cálculo de cada algoritmo (Tabela 8 e Tabela 9). O critério de aceitação foi o mesmo para os três: produzir uma atribuição e uma sequência válidas (rotas contíguas, sem atravessar prateleiras, e sem exceder a capacidade do robô selecionado).

Algoritmo	Custo total (s)	Tempo de cálculo	Desvio ao ótimo	Atribuições humano-sozinho	Observação
Exato	25,77	895 ms	0%	1 de 6	Ótimo (referência)
Guloso	31,78	3 ms	23,3%	1 de 6	Mais rápido; qualidade mais baixa
Meta-heurística	27,77	38 ms	7,8%	1 de 6	Perto do ótimo, muito mais rápida que o exato

Tabela 9 - Comparação dos três algoritmos (armazém 7×6, 6 recolhas, robô AMR Ligeiro).

Algoritmo	Custo total (s)	Tempo de cálculo	Desvio ao ótimo	Atribuições humano-sozinho	Observação
Exato	21,29	101 ms	0%	0 de 5	Ótimo (referência)

Guloso	21,29	14 ms	0%	0 de 5	Igualou o ótimo neste cenário
Meta-heurística	21,29	25 ms	0%	0 de 5	Igualou o ótimo neste cenário

Tabela 10 - Comparação dos três algoritmos num segundo layout (5×7, 5 recolhas, robô AMR Pesado).

Os dois cenários mostram o compromisso esperado: o algoritmo exato dá sempre o ótimo mas o seu tempo de cálculo cresce depressa com o número de recolhas; a heurística gulosa é sempre rápida, mas só encontrou o ótimo no segundo cenário, mais simples; a meta-heurística aproximou-se do ótimo em ambos, com um tempo de cálculo muito inferior ao do algoritmo exato. Em nenhum cenário a heurística ou a meta-heurística violaram a restrição de capacidade do robô: a recolha mais pesada do primeiro cenário (acima da capacidade do AMR Ligeiro) foi corretamente atribuída ao operador sozinho pelos três algoritmos.

13.6. Análise dos resultados

Os testes funcionais confirmaram o comportamento esperado em todos os casos, validando a segurança, o isolamento entre armazéns e a correção do controlo de stock e do fluxo de picking. Os testes ao ponto de encontro de uma única recolha confirmaram, em todos os cenários, rotas válidas e poupança não negativa para o operador. Os testes comparativos aos três algoritmos de planeamento confirmaram o padrão esperado na literatura (Petzoldt et al., 2023): o algoritmo exato garante o ótimo mas não escala; a heurística gulosa é rápida mas nem sempre ótima; a meta-heurística aproxima-se do ótimo num tempo muito inferior ao do algoritmo exato. Em todos os cenários, a restrição de capacidade do robô foi respeitada, confirmando que a atribuição humano-robô deixou de depender apenas da distância.

14. Sítio Web do Projeto

No cumprimento dos requisitos do projeto, foi disponibilizado um sítio web operacional que reúne toda a informação e os documentos do projeto. O sítio encontra-se publicado através do serviço GitHub Pages, associado ao repositório de código, e está acessível no seguinte endereço:

<https://luismmrafa-art.github.io/wms-projeto/>

A página apresenta a identificação do projeto e do autor, uma síntese do sistema desenvolvido, a descrição da arquitetura das três componentes e do algoritmo de coordenação humano-máquina, os resultados obtidos na validação e imagens das interfaces. Disponibiliza ainda ligações diretas para o repositório de código-fonte e para a versão em PDF do presente relatório.

A opção por uma página informativa, em detrimento da aplicação em execução acessível online, deveu-se à necessidade de garantir um endereço estável e permanente, independente da disponibilidade de um servidor de alojamento. A publicação através do GitHub Pages assegura, deste modo, que a informação do projeto permanece acessível de forma contínua.

15. Conclusões

15.1. Resultados obtidos

O projeto cumpriu os objetivos definidos na proposta, resultando no BoxToCar, uma prova de conceito funcional de picking colaborativo humano-máquina. Foram concretizadas as três componentes (API, backoffice web e aplicação móvel), a importação dinâmica de layouts e, sobretudo, um sistema de coordenação que já não se limita à distância: um catálogo de robôs com recomendação automática por layout, uma decisão multicritério que pondera tempo, peso e fragilidade, e três algoritmos de planeamento - exato, guloso e meta-heurística - comparados quanto ao tempo de cálculo e à qualidade da solução, e validações físicas (encaixe do artigo na prateleira, acessibilidade das prateleiras) que mantêm o modelo do armazém coerente com a realidade.

15.2. Dificuldades e soluções

As principais dificuldades incluíram a definição de um modelo de coordenação simultaneamente realista e computável - resolvida restringindo o robô ao corredor principal e reduzindo o problema a um caso do ponto de encontro sobre grelha -, o reforço iterativo da segurança para garantir o isolamento entre armazéns, e a compatibilização do código com o esquema da base de dados. Cada dificuldade foi ultrapassada e validada por testes.

15.3. Melhorias futuras

- Substituir a BFS por A* com custos diferenciados (corredores rápidos/lentos), e explorar outras meta-heurísticas - por exemplo, algoritmos genéticos - para conjuntos maiores de recolhas.
- Coordenar múltiplos operadores e robôs em simultâneo, com afetação dinâmica de tarefas.
- Suportar a importação de layouts em XML, além de JSON, e integrar dados reais de um ERP.
- Introduzir indicadores de desempenho (distância total, tempo, ocupação do robô) e relatórios.

15.4. Limitações

Sendo uma prova de conceito, o BoxToCar apresenta limitações que importa assumir. O robô é ainda virtual e único por armazém - a decisão sobre qual o tipo de robô operar é feita ao nível do armazém, e não numa frota heterogénea em simultâneo -, e não modela a navegação autónoma nem a prevenção de colisões em tempo real de múltiplos robôs físicos. O modelo de dados mantém uma simplificação consciente - cada unidade de stock é um registo distinto, o que não escala para grandes volumes -, mas as ligações entre produtos, encomendas e artigos passaram a fazer-se por chave, e as configurações continuam a assumir um único armazém por gestor. Do lado da segurança, embora as consultas sejam parametrizadas, uma cláusula LIMIT recorre a um valor convertido para inteiro em vez de parametrização direta, e o registo de operador passou a exigir um código de convite gerado pelo gestor. O algoritmo exato de planeamento só é viável para poucas recolhas (a implementação recusa-se a correr acima de 7), e a validação quantitativa incidiu sobre layouts de pequena e média dimensão. A validação de acessibilidade das prateleiras (secção 6.4) aplica-se à construção de novas prateleiras e à importação de plantas; não é retroativa a layouts já existentes que a precedam. Estas limitações não comprometem a demonstração do conceito e orientam o trabalho futuro (secção 15.3).

15.5. Contributo pessoal e aprendizagens

Este projeto permitiu integrar, num sistema completo, conhecimentos de desenvolvimento web e móvel, bases de dados, segurança e algoritmos, e ainda aprofundar um tema atual - a colaboração humano-máquina da Indústria 5.0. O desenho e a implementação do algoritmo de coordenação, bem como a sua validação quantitativa, constituíram a componente mais desafiante e enriquecedora, com um contributo determinante para a minha formação enquanto engenheiro informático.

Referências Bibliográficas

- Azadeh, K., De Koster, R., & Roy, D. (2019). Robotized and automated warehouse systems: Review and recent developments. *Transportation Science*, 53(4), 917–945. <https://doi.org/10.1287/trsc.2018.0873>
- Boysen, N., De Koster, R., & Weidinger, F. (2019). Warehousing in the e-commerce era: A survey. *European Journal of Operational Research*, 277(2), 396–411. <https://doi.org/10.1016/j.ejor.2018.08.023>
- Calzavara, M., Faccio, M., & Granata, I. (2023). Multi-objective task allocation for collaborative robot systems with an Industry 5.0 human-centered perspective. *The International Journal of Advanced Manufacturing Technology*, 128, 297–314. <https://doi.org/10.1007/s00170-023-11673-x>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tese de doutoramento, University of California, Irvine].
- Fragapane, G., De Koster, R., Sgarbossa, F., & Strandhagen, J. O. (2021). Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, 294(2), 405–426. <https://doi.org/10.1016/j.ejor.2021.01.019>
- Google. (2023). *Flutter documentation*. <https://docs.flutter.dev>
- Gualtieri, L., Rauch, E., & Vidoni, R. (2021). Emerging research fields in safety and ergonomics in industrial collaborative robotics: A systematic literature review. *Robotics and Computer-Integrated Manufacturing*, 67, Article 101998. <https://doi.org/10.1016/j.rcim.2020.101998>
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>
- Lackner, T., Hermann, J., Kuhn, C., & Palm, D. (2024). Review of autonomous mobile robots in intralogistics: State-of-the-art, limitations and research gaps. *Procedia CIRP*, 130, 930–935. <https://doi.org/10.1016/j.procir.2024.10.187>
- Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer*, 36(6), 47–56. <https://doi.org/10.1109/MC.2003.1204375>

Leng, J., Sha, W., Wang, B., Zheng, P., Zhuang, C., Liu, Q., Wuest, T., Mourtzis, D., & Wang, L. (2022). Industry 5.0: Prospect and retrospect. *Journal of Manufacturing Systems*, 65, 279–295. <https://doi.org/10.1016/j.jmsy.2022.09.017>

Löffler, M., Boysen, N., & Schneider, M. (2023). Human-robot cooperation: Coordinating autonomous mobile robots and human order pickers. *Transportation Science*, 57(4), 979–998. <https://doi.org/10.1287/trsc.2023.1207>

Maddikunta, P. K. R., Pham, Q. V., Prabadevi, B., Deepa, N., Dev, K., Gadekallu, T. R., Ruby, R., & Liyanage, M. (2022). Industry 5.0: A survey on enabling technologies and potential applications. *Journal of Industrial Information Integration*, 26, Article 100257. <https://doi.org/10.1016/j.jii.2021.100257>

Petzoldt, C., Harms, M., & Freitag, M. (2023). Review of task allocation for human-robot collaboration in assembly. *International Journal of Computer Integrated Manufacturing*, 36(11), 1675–1715. <https://doi.org/10.1080/0951192X.2023.220446>

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.

Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson.

Van Gils, T., Ramaekers, K., Caris, A., & de Koster, R. (2018). Designing efficient order picking systems by combining planning problems: State-of-the-art classification and review. *European Journal of Operational Research*, 267(1), 1–15. <https://doi.org/10.1016/j.ejor.2017.09.002>

Xu, X., Lu, Y., Vogel-Heuser, B., & Wang, L. (2021). Industry 4.0 and Industry 5.0 - Inception, conception and perception. *Journal of Manufacturing Systems*, 61, 530–535. <https://doi.org/10.1016/j.jmsy.2021.10.006>

Glossário

AMR - Robô móvel autônomo capaz de se deslocar num espaço para transportar mercadoria; neste projeto, modelado de forma virtual.

Ponto de encontro - Célula do corredor principal onde o operador entrega a carga ao robô, minimizando o transporte manual.

Picking - Processo de recolha física dos produtos de uma encomenda a partir das suas localizações.

Rendezvous - Problema de determinar um ponto de convergência entre dois ou mais agentes com origens distintas.

BFS - Algoritmo de procura em largura que determina o caminho mais curto em número de passos numa grelha uniforme.

Token - Credencial digital assinada, emitida no início de sessão, que identifica e autoriza o utilizador.

WMS - Sistema informático de gestão das operações de um armazém.

Apêndices

Apêndice A - Estrutura de rotas da API

POST	/api/registo	(publico)	cria armazem + gestor
POST	/api/operador/registo	(publico)	cria operador num armazem
POST	/api/login	(publico)	devolve token JWT
GET	/api/inventario	(token)	prateleiras e produtos
POST	/api/produtos/novo	(token)	entrada de produto
POST	/api/prateleiras/nova	(token)	cria prateleira
GET	/api/tarefas/pendentes	(token)	tarefas do operador
POST	/api/operador/concluir	(token)	confirma recolha
POST	/api/encomendas/carrinho	(token)	gera ordens de picking
POST	/api/armazem/importar	(token)	importa planta (JSON)
GET	/api/coordenacao/plano	(token)	ponto de encontro + rotas

Apêndice B - Esquema das tabelas da base de dados

Armazens(ID, Nome, Cidade, LarguraCorredorM, RoboTipoID,CodigoConvite, PrateleiraLarguraCm, PrateleiraProfundidadeCm, PrateleiraAlturaNivelCm)
Usuarios(ID, Nome, Email, Senha, Cargo, ArmazemID)
Prateleiras(ID, PosX, PosY, Niveis, ArmazemID)
Produtos(ID, Nome, PrateleiraID, Nivel, ArmazemID, ArtigoID)
Artigos(ID, ArmazemID, Nome, PesoKg, ComprimentoCm, LarguraCm, AlturaCm, Fragil)
Encomendas(ID, ProdutoNome, ArtigoID, Quantidade, Estado, Data, ArmazemID)

Apêndice C - Pseudocódigo dos algoritmos de coordenação e planeamento

Núcleo geométrico (coordenacao.js), reutilizado pelos três algoritmos:

```
função calcularDistanciasTarefa(prateleiras, grelha, alvo, depósito):  
  ocupadas <- {células das prateleiras}  
  acessos <- vizinhos livres de alvo  
  distA <- BFS(origens = acessos, ocupadas)  
  encontro <- célula do perímetro que minimiza distA  
  distDep <- BFS(origem = depósito, ocupadas)  
  devolver {encontro, distA[encontro], distDep[encontro], distA[depósito]}
```

Decisão multicritério de uma tarefa (custoDecisao.js):

```
função custoTarefa(atribuição, distâncias, robô, artigo):  
  se atribuição = humano_sozinho ou robô inexistente:  
    devolver tempo(distA[depósito], velocidadeHumano)  
  tempoOperador <- tempo(distA[encontro], velocidadeHumano)  
  tempoRobo <- tempo(distDep[encontro], velocidadeRobo)  
  se artigo.fragil: tempoRobo <- tempoRobo × 1,35  
  devolver 0,7 × tempoOperador + 0,3 × tempoRobo  
  
função avaliarTarefa(distâncias, robô, artigo):  
  se peso(artigo) > capacidade(robô): devolver humano_sozinho // inviável  
  devolver a atribuição (humano_sozinho | humano_robô) de menor custoTarefa
```

Os três algoritmos de planeamento (algoritmos.js) - atribuem e ordenam um conjunto

de tarefas:

```
função resolverExato(tarefas, contexto):  
  para cada ordem em permutações(tarefas):  
    para cada atribuição em combinações_viáveis(tarefas):  
      custo <- avaliarSequencia(ordem, atribuição, contexto)  
      guarda o melhor (ordem, atribuição, custo)  
  devolve o melhor  
  
função resolverGuloso(tarefas, contexto):  
  posição <- depósito  
  enquanto houver tarefas por atribuir:  
    próxima <- tarefa pendente mais próxima de posição  
    atribuição[próxima] <- avaliarTarefa(distâncias(próxima), robô, artigo(próxima))  
    posição <- ponto de encontro ou depósito, conforme a atribuição  
  devolve (ordem, atribuição)  
  
função resolverMetaheuristica(tarefas, contexto):  
  solução <- resolverGuloso(tarefas, contexto)  
  temperatura <- T0  
  repetir N vezes:  
    vizinho <- trocar ordem(2 tarefas) OU inverter atribuição(1 tarefa, se viável)  
    Δ <- custo(vizinho) - custo(solução)  
    se Δ < 0 ou aleatório() < exp(-Δ / temperatura): solução <- vizinho  
    temperatura <- temperatura × arrefecimento  
  devolve a melhor solução encontrada
```

Apêndice D - Declaração de utilização de ferramentas de inteligência artificial

Apresenta-se em seguida o detalhe da utilização de ferramentas de inteligência artificial no desenvolvimento deste projeto, conforme declarado na secção 1.4. Os prompts são transcritos tal como foram submetidos.

D.1 - Aprendizagem de Git e GitHub Ferramenta e versão: Claude Code, versão 2.1.190

Objetivo: Compreender os conceitos de controlo de versões e o fluxo de trabalho do Git e do GitHub, sem experiência prévia na ferramenta.

Prompt:

Atua como um professor de Engenharia de Software. Sou novo no uso de controlo de versões e preciso de um guia para iniciantes, passo a passo, sobre como utilizar o GitHub.

Por favor, explica-me:

- 1. Os conceitos básicos: o que é um repositório, um commit e um push.*
- 2. Como usar a aplicação oficial (GitHub Desktop) para conectar o meu código local ao repositório na nuvem, desde criar o repositório até enviar o código.*
- 3. Como mandar o código para lá através do terminal (comandos básicos de Git), caso eu prefira não usar a aplicação visual.*

Usa uma linguagem simples, dá exemplos práticos e foca-te no fluxo de trabalho diário mais comum (alterar código -> guardar -> enviar para o GitHub).

Resultado obtido: Explicação dos conceitos de repositório, commit e push, e do fluxo de trabalho diário, tanto pela aplicação GitHub Desktop como pelo terminal.

Verificação do autor: Os comandos foram executados pelo autor e o resultado confirmado diretamente no repositório remoto.

D.2 - Automatização de commits e envio para o repositório

Ferramenta e versão: Claude Code, versão 2.1.190

Objetivo: Gerar mensagens de commit descritivas, segundo o padrão Conventional Commits, e enviar as alterações para o repositório remoto.

Prompt:

A tua função é atuar como um Agente de Integração de Código.

Recebeste as alterações de código mais recentes.

Executa as seguintes tarefas automaticamente:

- 1. Análise de Código: Verifica as alterações introduzidas para garantir que não existem erros críticos de sintaxe que quebrem a compilação.*
- 2. Geração de Commit Semântico: Analisa o diff das alterações e gera automaticamente uma mensagem de commit seguindo o padrão de Conventional Commits (ex: `feat: adiciona login`, `fix: corrige erro de base*

de dados`, `refactor: otimiza função`)). A mensagem deve ser em português, clara e descritiva.

3. Push Direto: Executa o commit com a mensagem gerada e faz o push imediato destas alterações para o repositório remoto na branch atual.

Condição de Falha: Se o código contiver erros óbvios que inviabilizem a execução do sistema, interrompe o processo, não faça o commit nem o push, e regista o erro no log do pipeline.

Resultado obtido: Mensagens de commit geradas automaticamente em português. A ferramenta acrescenta, por defeito, uma atribuição de coautoria (Co-Authored-By: Claude) que fica registada no histórico público do repositório.

Verificação do autor: Embora o prompt determinasse o envio imediato, a ferramenta solicita autorização antes de executar cada comando. O autor verificou a mensagem de commit gerada e as alterações correspondentes antes de autorizar o commit e o envio para o repositório remoto.